

BullEye AI

Optimiser sa stratégie DCA grâce au Deep Learning

Retour d'expérience complet sur 12 mois de recherche

Avril 2025 — Avril 2026

Jean Frégeville

Centrale Lille

Projet académique — Défi G1G2

Rédigé en mai 2026

Résumé

Ce document retrace l'entièreté de mon projet **BullEye AI**. L'objectif est de concevoir un système d'investissement périodique intelligent. Autrement dit, le but est de développer un modèle de Machine Learning permettant d'effectuer un DCA (*Dollar Cost Averaging*) optimisé.

Ce compte rendu documente non seulement les succès techniques, mais aussi les erreurs, les impasses et les enseignements tirés.

Table des matières

1	Contexte et motivation du projet	4
1.1	Le départ	4
1.2	Problématique	4
1.3	Les objectifs	4
1.4	Les actifs ciblés	5
2	Théorie simple du DCA augmenté	6
2.1	Formalisation des Méthodes	6
2.1.1	Stratégie S_1 : DCA Classique	6
2.1.2	Stratégie S_2 : DCA Prédictif (IA)	6
2.2	Analyse Différentielle	7
2.3	Précisions sur ΔN	7
2.4	Dans quel cas la stratégie IA est la plus efficace ?	8
2.5	Limites de la Stratégie	8
2.6	Résultats	9
2.7	Analyse	10
2.7.1	Exemple Numérique : Cycle de 4 Périodes	10
2.7.2	Comparaison des Stocks Finalisés	11
2.7.3	Interprétation Mathématique de la Faible Performance	11
3	Recherches bibliographiques	13
3.1	Point de départ : un modèle issu d'un électif	13
3.2	Méthode de recherche adoptée	13
3.3	Pourquoi utiliser des modèles de Deeplearning plutôt que des modèles de ML classiques ?	14
3.3.1	Les 3 propriétés fondamentales des séries temporelles	14
3.4	Revue rapide des différents modèles qui sont abordés dans ce rapport	15
3.4.1	Le point de départ : Le Perceptron Multicouche (MLP)	15
3.4.2	L'introduction de la mémoire : Le Réseau Récurrent (RNN)	15
3.4.3	La solution : le LSTM	16
3.4.4	La simplification : GRU	17
3.4.5	Le Changement de Paradigme : Les TCN (Temporal Convolutional Networks)	19
4	Sélection du modèle optimal	22
4.1	Architecture du Pipeline de Prédiction par LSTM	22
4.2	Méthodologie de comparaison itérative	23
4.3	Benchmark Exhaustif des Architectures Deep Learning	24
4.3.1	Justification de l'approche Benchmark	24
4.4	Résultats	24
4.4.1	Phase 1 : Exploration globale (le benchmark large)	24
4.4.2	Phase 2 : Affinement avec mécanismes d'attention et TCN	25
4.4.3	Phase 3 : Test de robustesse	26
4.4.4	Phase 4 : Dernière comparaison et verdict final	26
4.5	L'architecture gagnante : Attention + TCN + BiGRU	27
4.6	Biais critique découvert durant cette phase	28

5	Évaluation des limites prédictives	29
5.1	Démarche intellectuelle et objectif du test	29
5.2	Visualisation de la dégradation des performances	29
5.3	Analyse	31
6	Modèles de classification et enrichissement des données	33
6.1	Pivot stratégique : de la régression vers la classification	33
6.2	Impact de la réduction du nombre d'indicateurs en entrée	34
7	Pipeline de classification final et résultats	38
7.1	De la barrière statique à la barrière dynamique	38
7.2	Enrichissement des données et pipeline technique	39
7.3	Benchmark des 9 architectures	39
7.4	Justification du choix de l'architecture ATG	41
7.4.1	Définition du F_1 -Score, de la Précision et du Rappel	41
7.4.2	L'illusion du Deep-Dense	41
7.4.3	Analyse de la Matrice de Confusion	42
7.5	Généralisation du modèle	43
7.5.1	Tests de généralisation	44
7.5.2	Résultats	45
8	Enseignements et pistes d'amélioration	46
8.1	La limite structurelle des données passées	46
8.2	Ce que cela implique pour BullEye AI	46
8.3	Pistes d'amélioration identifiées	47
9	Conclusion	48
10	Annexes	49
10.1	Annexe : pipeline générique LSTM	49
10.2	Annexe 2 : Benchmark 5 modèles	51
10.3	Annexe 3 : dictionnaire détaillé des 17 features du pipeline final	55

1. Contexte et motivation du projet

1.1. Le départ

Ma volonté première au travers de ce défi était de coder mon premier modèle de Machine Learning. Je n'avais quasiment aucune notion de Machine Learning en avril 2025 et souhaitais donc me lancer car le domaine représente selon moi un indispensable pour les années à venir.

J'ai donc initialement passé beaucoup de temps (avril 2025 - août 2025) à explorer le domaine sans tellement savoir vers où m'orienter.

Le point de bascule se fera suite à un cours de Deep Learning à Centrale au cours duquel j'ai développé un modèle de prédictions boursières qui semblait fonctionner particulièrement bien. Ce sujet me semblait être un bon moyen d'associer machine learning et finance, donc je me suis lancé.

1.2. Problématique

Ce projet s'insère dans le cadre du **défi G1G2**, module intégré au cursus de Centrale Lille afin de nous permettre de mener à bien un projet durant 12 mois entre la première et la deuxième année d'école. Mon choix c'est tout de suite porté vers le domaine de la finance de marché qui me passionne depuis que j'ai commencé à investir.

L'idée fondatrice de BullEye AI repose sur un constat simple : **l'investisseur passif est gagnant sur le long terme**. Ce que j'appelle investisseur passif est une personne investissant à long terme selon une stratégie essentiellement basée sur du "buy and hold" (= acheter et garder ses positions).

Une des informations clés en investissement est que les marchés financiers tendent à croître structurellement, notamment en raison de l'augmentation de la masse monétaire mondiale (*M2 supply* - voir le **graphique 1**). La stratégie DCA - investir une somme fixe à intervalles réguliers - est reconnue comme l'une des approches les plus robustes pour un profil investisseur long terme.

A partir de ce constat, je me suis demandé dans quelle mesure *il est possible de maximiser ses gains tout en conservant ce profil d'investisseur patient, simplement en choisissant mieux ses moments d'entrée sur le marché ?*

BullEye AI est ma tentative de réponse.

1.3. Les objectifs

Les objectifs développés dans ce document sont :

1. Formaliser une théorie simple de DCA augmenté par un modèle prédictif ;
2. Prouver l'efficacité de cette théorie en pratique ;
3. Développer plusieurs modèles de machine learning pour prédire les cours boursiers ;
4. Evaluer ces modèles, les comparer et choisir le meilleur ;
5. Généraliser le modèle sélectionné à d'autres actifs que l'actif d'entraînement.

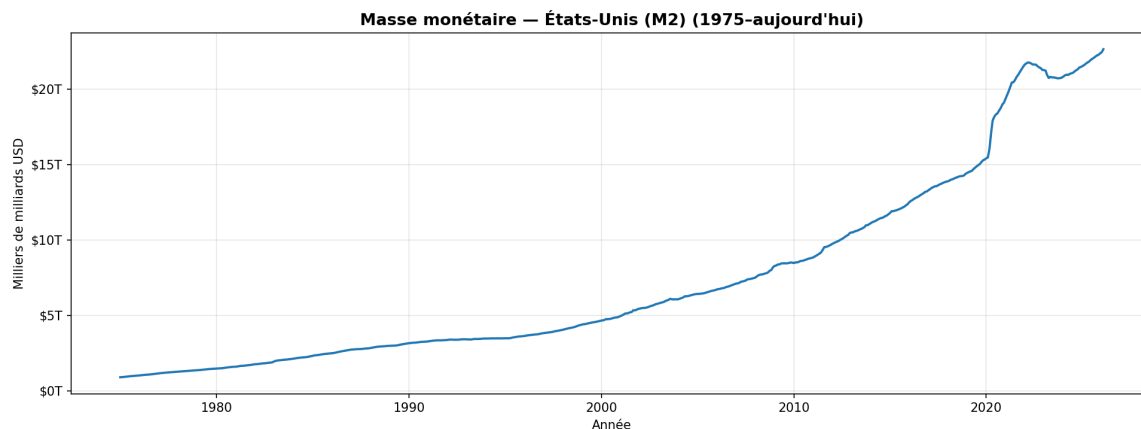


FIGURE 1 – Évolution de la masse monétaire M2 des Etats-Unis depuis 1975 | Source : Masse_Monétaire.ipynb

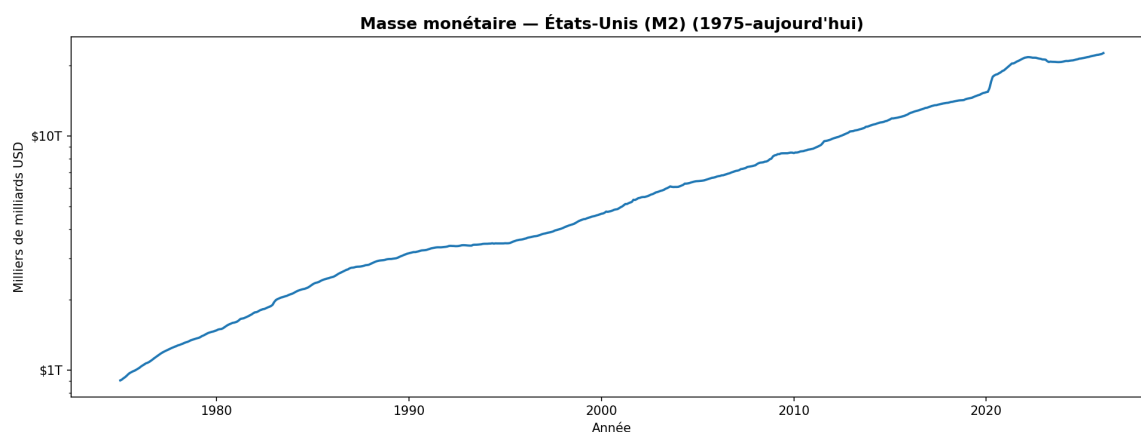


FIGURE 2 – Évolution de la masse monétaire M2 des Etats-Unis depuis 1975 - version logarithmique | Source : Masse_Monétaire.ipynb

1.4. Les actifs ciblés

Les trois principaux actifs retenus pour les tests sont le **S&P 500** (ticker = SPY), le **Nasdaq** (ticker = QQQ) et le **Bitcoin** (ticker = BTC-USD).

Les raisons de ces choix d'actifs sont les suivantes :

1. Historique de données important ;
2. Facilement accessibles via `yfinance` ;
3. Volatilités (nettement) différentes.

Je me laisse la possibilité d'utiliser d'autres actifs au besoin par exemple lors de l'établissement de la théorie du DCA augmenté qui demande des actifs dont la valeur diffère grandement.

2. Théorie simple du DCA augmenté

Cette étude porte sur la comparaison du nombre de parts T_k accumulés via deux méthodes d'investissement.

Supposons un investissement fixe M à chaque intervalle t . Autrement dit, notre investisseur fictif investit M euros à partir d'une date t_0 puis à un intervalle régulier de t , créant ainsi la suite des temps d'investissement t_k . Posons également P_k et N_k respectivement le prix d'un actif quelconque ainsi que le nombre de parts achetées à l'instant t_k .

L'objectif est de quantifier l'avantage d'une connaissance parfaite du prix P_{k+1} à l'instant t_k par rapport à un investissement aveugle (i.e. investir M tous les t_k).

2.1. Formalisation des Méthodes

Soit k un entier fixé, correspondant à un temps d'investissement quelconque de l'investisseur.

Stratégie S_1 : DCA Classique

Le nombre de parts d'actifs achetées à chaque étape est invariant par rapport à la tendance puisque nous ne la connaissons pas :

$$N_{k,S_1} = \frac{M}{P_k} \quad (1)$$

Ainsi, l'investisseur investit dans l'actif de manière régulière, toujours avec la même intensité. C'est le principe du DCA.

Stratégie S_2 : DCA Prédicatif (IA)

Supposons que nous avons une IA parfaite qui connaît sans erreur, au temps t_k , le prix de l'actif à l'instant t_{k+1} .

Alors, nous considérons dans un premier temps que l'IA adopte un comportement binaire basé sur P_{k+1} :

★ **Si** $P_{k+1} \geq P_k$: Achat immédiat au prix P_k .

$$N_{k,S_2} = \frac{M}{P_k}$$

★ **Si** $P_{k+1} < P_k$: Report de l'achat à l'instant $k + 1$.

$$N_{k,S_2} = 0 \quad \text{et} \quad N_{k+1,S_2} = \frac{2M}{P_{k+1}}$$

2.2. Analyse Différentielle

Sur une période où le prix monte, la différence de parts d'actifs est évidemment nulle car la stratégie IA réplique celle du DCA (l'IA achète maintenant car le prix sera plus cher au coup d'après).

Sur une période où le prix baisse ($P_{k+1} < P_k$), la différence de parts d'actifs accumulées ΔT est :

$$\Delta N = N_{S_2} - N_{S_1} = \frac{2M}{P_{k+1}} - \left(\frac{M}{P_k} + \frac{M}{P_{k+1}} \right) \quad (2)$$

En simplifiant, nous obtenons l'écart fondamental :

$$\Delta N = M \left(\frac{1}{P_{k+1}} - \frac{1}{P_k} \right) = M \frac{P_k - P_{k+1}}{P_k P_{k+1}} \quad (3)$$

Et ce qui nous intéresse, c'est cette inégalité :

$$\Delta N > 0 \quad (4)$$

qui découle de la stricte positivité de M , des P_k pour tout k et du fait que nous ayons considéré $P_{k+1} < P_k$.

Autrement dit, nous accumulons plus d'actifs avec la méthode IA !

2.3. Précisions sur ΔN

Comprenons bien que la stricte positivité de ΔN est intéressante, certes, mais nous aimerions savoir dans quel cas notre stratégie IA est la plus optimale.

Pour ce faire, étudions en détail les variables qui composent ΔN .

Remarquons tout d'abord que cette stratégie est fortement liée à l'existence d'entiers k de sorte que $P_{k+1} < P_k$. Autrement dit, plus le nombre de d'entiers k tels que $P_{k+1} < P_k$ sont nombreux, plus les cas favorables à la stratégie sont nombreux et donc plus nous pouvons creuser l'écart avec un DCA classique.

L'autre point essentiel est la **volatilité** σ : Plus l'amplitude $|P_k - P_{k+1}|$ est forte, plus l'IA surperforme le DCA.

CONCLUSION SUR L'OPTIMALITÉ

Il est essentiel de distinguer deux types d'optimisation pour ne pas mal interpréter l'impact des variables :

- **L'Accumulation Absolue** (ΔN) : Nombre brut de parts supplémentaires. Proportionnel à M et inversement proportionnel au prix P .
- **La Performance Relative** (η) : Ratio de surperformance. **Indépendant** de M et de P .

Démonstration de l'invariance par le montant M :

Soit η le différentiel de performance relative entre la stratégie IA et le DCA classique. En reprenant l'expression de la surperformance sur un cycle de baisse ($P_{k+1} < P_k$) :

$$\eta = \frac{\Delta N}{N_{S1}} = \frac{M \left(\frac{1}{P_{k+1}} - \frac{1}{P_k} \right)}{\frac{M}{P_k} + \frac{M}{P_{k+1}}} \quad (5)$$

En factorisant par M au numérateur et au dénominateur, puis en réduisant au même dénominateur les termes entre parenthèses :

$$\eta = \frac{M \cdot \left[\frac{P_k - P_{k+1}}{P_k P_{k+1}} \right]}{M \cdot \left[\frac{P_k + P_{k+1}}{P_k P_{k+1}} \right]} \quad (6)$$

Après simplification par M et par le produit des prix ($P_k P_{k+1}$), on obtient l'expression simplifiée de la rentabilité relative :

$$\eta = \frac{P_k - P_{k+1}}{P_k + P_{k+1}} \quad (7)$$

Conclusion : Le rendement relatif η ne dépend que de la dynamique des prix (la volatilité). Le montant M s'annule systématiquement dans le calcul du ratio : investir 100€ ou 1000€ produit exactement la même courbe de rentabilité en pourcentage, ce qui explique les résultats obtenus lors des simulations numériques.

2.4. Dans quel cas la stratégie IA est la plus efficace ?

Ces éléments découlent directement du paragraphe précédent :

- ★ **Période de DCA faible** (cela augmente la probabilité de tomber sur des k tq $P_{k+1} < P_k$) ;
- ★ **Volatilité σ importante de l'actif.**

2.5. Limites de la Stratégie

Bien que la supériorité mathématique de la stratégie IA soit démontrée ($\Delta N \geq 0$), son efficacité réelle dépend de la morphologie de la courbe des prix.

Problème de la directionnalité unilatérale

Dans un scénario de **Bull Run pur** (hausse séquentielle sans aucune bougie de correction), les deux stratégies convergent vers une efficacité identique. Si pour tout k , $P_{k+1} > P_k$, alors l'IA exécute l'achat immédiatement à chaque période, simulant

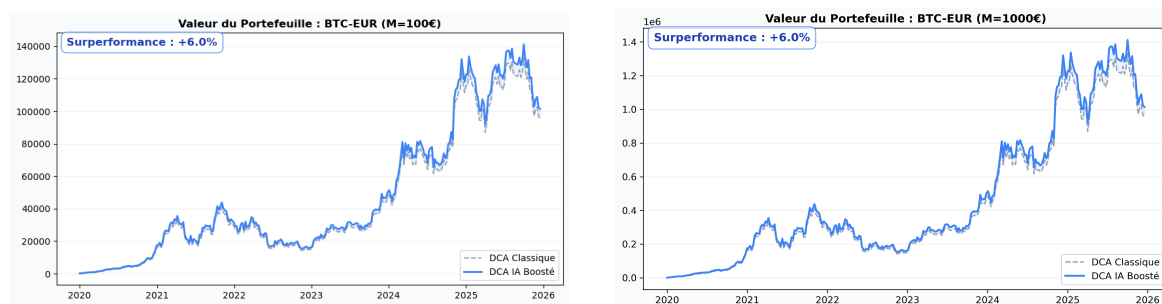
exactement un DCA classique. L'avantage prédictif est alors nul car il n'existe aucune opportunité de report d'achat favorable.

Autrement dit, le modèle IA est sensible à la **densité des points de pivot** et ne crée de la valeur que sur les occurrences où la dérivée du prix change de signe. Sur un marché très directionnel, ces points sont rares.

2.6. Résultats

Les simulations numériques suivantes illustrent concrètement l'impact de la volatilité et du sous-jacent sur l'efficacité de la stratégie IA.

Dans un premier temps, j'ai voulu vérifier empiriquement la démonstration mathématique précédente stipulant que la surperformance relative (η) est invariante par rapport au capital investi (M). Pour ce faire, j'applique tout simplement la théorie augmentée sur le Bitcoin (BTC-EUR) avec deux montants de DCA différents.



BTC-EUR ($M = 100\text{€}$)

BTC-EUR ($M = 1000\text{€}$)

FIGURE 3 – Preuve empirique de l'invariance. Malgré un capital investi multiplié par 10, l'algorithme génère exactement le même ratio de surperformance (+6.0%). La stratégie optimise le point d'entrée, indépendamment de la taille de la position.

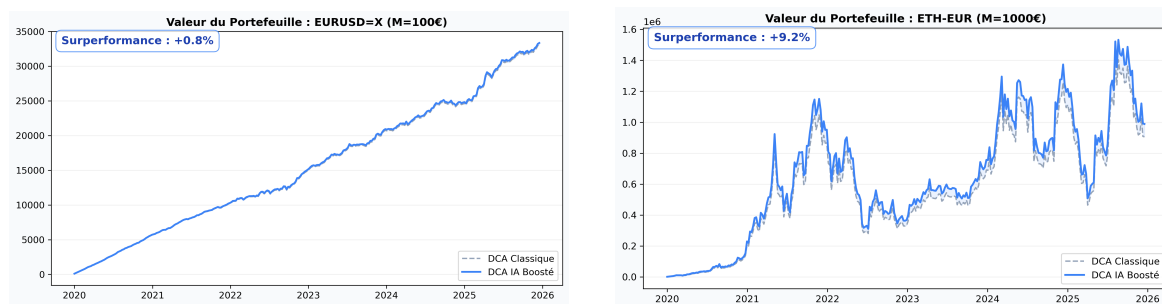
Une fois cette invariance confirmée, nous pouvons confronter l'Axiome de l'IA Prédictive à la réalité de différents marchés. J'ai pour cela sélectionné quatre actifs aux profils de volatilité diamétralement opposés.



S&P 500 : +1.5%

SNDL : +10.5%

FIGURE 4 – Contraste sur le marché Actions ($M = 100\text{€}$). À gauche, le S&P 500, indice lissé et très directionnel (Bull Market), offre peu de corrections : l'IA surperforme à peine. À droite, l'action SNDL, extrêmement volatil, permet à l'IA de capturer +10.5% d'Alpha en exploitant les forts replis.



EUR/USD : +0.8% ($M = 100\text{€}$)

Ethereum : +9.2% ($M = 1000\text{€}$)

FIGURE 5 – Contraste entre classes d'actifs. Le Forex (EUR/USD), marché le plus efficient et le moins volatil au monde, rend la capture d'anomalies quasi impossible. À l'inverse, l'Ethereum offre des vagues de volatilité massives dont le modèle se nourrit pour créer un écart significatif avec le DCA classique.

Ces résultats valident formellement l'axiome initial : le modèle de DCA boosté par IA n'est pas un outil miracle universel. Il agit comme un **filet à volatilité** dont les mailles ne retiennent la valeur que si le marché ondule suffisamment (Range ou forte volatilité) pour créer des inefficiences de court terme.

2.7. Analyse

Comme vu précédemment, l'expérimentation numérique sur des actifs réels montre une surperformance relative entre 1% et 11% sur 6 ans. Pour comprendre pourquoi ce chiffre n'est pas plus élevé malgré une anticipation parfaite, analysons un cycle d'investissement simplifié.

Exemple Numérique : Cycle de 4 Périodes

Supposons un investissement de $M = 100 \text{ €}$ par période sur un actif volatil.

Période (k)	Prix (P_k)	Tendance (P_{k+1})	Action S1	Action S2 (IA)
1	100 €	Hausse	Achète 1.00 u	Achète 1.00 u
2	110 €	Hausse	Achète 0.91 u	Achète 0.91 u
3	120 €	Baisse	Achète 0.83 u	Report (0 u)
4	90 €	-	Achète 1.11 u	Achat 2M (2.22 u)

Comparaison des Stocks Finalisés

Le nombre total de parts accumulées à l'issue du cycle est :

- **DCA Classique** (N_{S_1}) : $1.00 + 0.91 + 0.83 + 1.11 = 3.85$ unités.
- **DCA IA** (N_{S_2}) : $1.00 + 0.91 + 0 + 2.22 = 4.13$ unités.

La surperformance relative pour ce cycle est de :

$$\eta = \frac{4.13 - 3.85}{3.85} \approx 7.27\% \quad (8)$$

Interprétation Mathématique de la Faible Performance

Pourquoi l'IA ne gagne-t-elle que 7% alors qu'elle a optimisé l'achat de la période 3 avec un gain de +33% (achat à 90 au lieu de 120) ?

1. **Effet de Dilution Temporelle** : Le gain de 33% ne s'applique que sur le flux de la période 3 (M). Les unités déjà acquises lors des périodes 1 et 2 représentent une part majoritaire du portefeuille et ne bénéficient pas de l'optimisation locale.
2. **Poids du Passé** : Soit N_{total} le nombre de parts accumulés. L'impact d'une décision optimale à l'instant k est donné par :

$$\Delta\eta_{global} = \frac{\Delta N_k}{N_{total}} \quad (9)$$

Plus le temps passe, plus N_{total} augmente, et plus l'impact relatif de chaque "bon coup" de l'IA diminue.

L'Axiome de l'IA Prédictive

La stratégie IA est une stratégie de **capture de volatilité**. Son alpha (surperformance) est maximal dans un marché latéral*("Range") et minimal dans un marché à forte tendance ("Trending Market"). En d'autres termes, l'IA ne bat le DCA que si le marché lui offre des erreurs de prix à corriger.

(*) Un marché latéral (souvent appelé range, marché en consolidation ou flat market en anglais) est une phase où le prix d'un actif (action, crypto, matière première) évolue dans un canal horizontal, sans tendance claire à la hausse ou à la baisse. Ainsi, nous ne sommes pas dans le cas d'un bullrun, et il y a donc un changement de signe de la dérivée ce qui permet à la stratégie augmentée de surperformer le DCA classique.

CONCLUSION SUR L'OPTIMALITÉ

La stratégie IA est une **optimisation à la marge**. Elle est extrêmement efficace pour réduire le prix de revient moyen (PRU) au début de l'investissement, mais son influence décroît asymptotiquement à mesure que le stock d'actifs accumulés devient important face au flux d'investissement périodique M .

Finalement, le plus optimal serait d'utiliser le modèle pour faire du trading de haute fréquence car cela augmente la probabilité d'obtenir des entiers k qui assure une baisse au coup $k+1$. Le problème (et nous le verrons) est que les modèles d'IA sont de moins en moins bon quand on diminue la période de prédiction... Il faudra donc prendre le modèle le plus performant pour une période de prédiction minimale donnée !

3. Recherches bibliographiques

3.1. Point de départ : un modèle issu d'un électif

Au lancement du projet, le modèle de prédiction utilisé était le fruit d'un cours suivi à Centrale. Les résultats des modèles ainsi développés étaient à priori très encourageants et m'ont donné envie d'aller plus loin (voir le **graphique 6**).

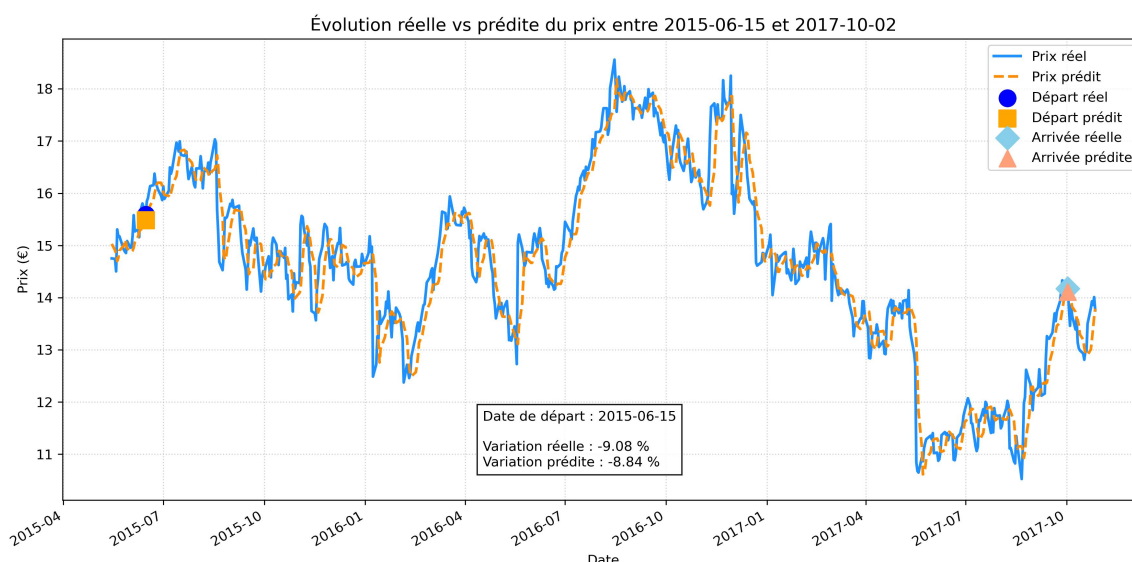


FIGURE 6 — Plan d'investissement long terme | Source : Plan_Investissement_CNN.ipynb

Explication du graphique 6 : Ce graphe présente l'évolution entre le point d'entrée réel/prédit et le point de sortie réel/prédit. L'objectif du projet de ce cours était de montrer qu'un modèle de machine learning était capable de suivre l'évolution de cours boursiers. Cependant, ce résultat à priori très positif n'est que la conséquence d'une prédiction très court terme (à 1 jour) - le modèle n'apprend pas, il ajuste simplement le prix de demain en fonction du prix du jour même. Cette réflexion sera approfondie plus tard car je ne m'en suis rendu compte que durant mon défi !

L'objectif de cette première phase était d'identifier des architectures plus performantes en m'appuyant sur la littérature scientifique existante.

3.2. Méthode de recherche adoptée

Plutôt que de lire intégralement des papiers de recherche, j'ai adopté une approche pragmatique : fournir des articles [1] [2] [3] [4] [5] [6] [7] issus de Google Scholar à **Perplexity**, en lui demandant d'en extraire les architectures les plus prometteuses pour la prédiction de cours boursiers.

Les résultats de la requête peuvent être résumés ainsi :

Synthèse bibliographique — Conclusions clés :

- **LSTM & GRU** : efficaces pour les dépendances temporelles longues ; le GRU est ~30% plus rapide que le LSTM pour des performances comparables (voir abstract de [2] pour la rapidité et la conclusion de [3] pour les performances comparables).
- **TCN (Temporal Convolutional Networks)** : surpassent souvent les RNN sur les tâches multi-horizon grâce aux convolutions dilatées parallélisables (voir la conclusion page 8 de [4]).
- **MLP (Dense Networks)** : insuffisants seuls pour les séries non-stationnaires, mais utiles en tête prédictive (voir l'abstract de [8]).
- **Stratégie multi-horizon** : la stratégie Direct offre généralement un meilleur compromis biais/variance que la stratégie Recursive sur les horizons courts à moyens (voir la conclusion de [6]).

mcm

Avant de passer à la vérification de ces affirmations et au choix du modèle le plus performant, j'ai souhaité passer quelques temps sur la compréhension théorique de ces modèles, car les conclusions issues de ces papiers ne sont pas anodines : tout découle de la structure même de ces modèles.

3.3. Pourquoi utiliser des modèles de Deeplearning plutôt que des modèles de ML classiques ?

Ma première question a été de savoir pourquoi ne pas utiliser de simples modèles de ML. La réponse est simple : la plupart des algorithmes classiques (Régression Linéaire simple, Random Forest, k-NN) reposent sur une hypothèse forte : les données sont **I.I.D.** (Indépendantes et Identiquement Distribuées).

Or, les séries temporelles violent ces deux principes.

Les 3 propriétés fondamentales des séries temporelles

1. L'ordre est une information (Séquentialité)

- Séquence A : 100\$ $\rightarrow$ 105\$ $\rightarrow$ 110\$ (Tendance haussière).
- Séquence B : 110\$ $\rightarrow$ 100\$ $\rightarrow$ 105\$ (Volatilité/Baisse).

Les valeurs sont les mêmes, mais l'histoire (et donc la prédiction future) est totalement différente.

2. Non-Stationnarité

C'est la propriété la plus embêtante. Une série est "non-stationnaire" si ses statistiques (moyenne, variance) changent au cours du temps.

- *Exemple* : Le comportement du Bitcoin en 2013 (très peu liquide, variations énormes) n'est plus le même qu'en 2024 (plus institutionnel).
- *Conséquence* : Un modèle entraîné sur 2013 échouera à prédire 2024 car "les règles du jeu ont changé".

3. Motifs Multi-Échelles

Les séries temporelles contiennent des cycles imbriqués :

- **Micro-structure** : Bruit minute par minute.
- **Saisonnalité** : Cycles hebdomadaires (crypto active le week-end) ou mensuels.
- **Tendance (Trend)** : Mouvement global sur l'année.

Les modèles classiques peinent à séparer ces fréquences.

3.4. Revue rapide des différents modèles qui sont abordés dans ce rapport

Cette section détaille l'évolution des architectures neuronales, en expliquant les blocages mathématiques qui ont poussé les chercheurs à concevoir des modèles de plus en plus sophistiqués.

Le point de départ : Le Perceptron Multicouche (MLP)

Le Perceptron Multicouche (MLP) est l'architecture de base du Deep Learning. À chaque instant t , il reçoit une entrée $\mathbf{x}_t$ (par exemple, le prix du jour) et applique une transformation matricielle suivie d'une fonction d'activation non-linéaire σ (voir [9] pour plus de détails) :

$$\mathbf{h}_t = \sigma(\mathbf{W}\mathbf{x}_t + \mathbf{b}) \quad (10)$$

Le blocage structurel : Le MLP souffre d'**amnésie temporelle**. L'équation ci-dessus montre que le calcul de $\mathbf{h}_t$ dépend uniquement de $\mathbf{x}_t$. Le modèle traite les jours sous l'hypothèse qu'ils sont indépendants (hypothèse i.i.d.). Si on mélange l'ordre des jours dans notre jeu de données, le MLP donnera exactement les mêmes prédictions.

Transition : Pour comprendre une tendance, il faut se souvenir du passé. Il a donc fallu créer une architecture capable de faire transiter une information de l'instant $t - 1$ à l'instant t . C'est la naissance des RNN (voir [10] pour comprendre la transition).

L'introduction de la mémoire : Le Réseau Récurrent (RNN)

Le Réseau de Neurones Récurrents (RNN) introduit une boucle temporelle. L'état caché du réseau ($\mathbf{h}_t$) ne dépend plus seulement de l'entrée du jour ($\mathbf{x}_t$), mais aussi de **l'état de la veille** ($\mathbf{h}_{t-1}$) :

$$\mathbf{h}_t = \tanh(\mathbf{W}_x\mathbf{x}_t + \mathbf{W}_h\mathbf{h}_{t-1} + \mathbf{b}) \quad (11)$$

Le problème mathématique (Vanishing Gradient) : Bien que logique, le RNN standard échoue en pratique sur de longues séquences temporelles. Lors de l'apprentissage, l'algorithme rétropropage l'erreur dans le temps. Pour comprendre l'impact de $t - 100$ sur t , l'algorithme multiplie 100 fois la matrice de poids $\mathbf{W}_h$. Si ces poids sont inférieurs à 1, le gradient s'écrase exponentiellement vers zéro [11]. Le réseau devient incapable de relier une cause lointaine à une conséquence présente, ce qui est rédhibitoire pour notre cas d'usage.

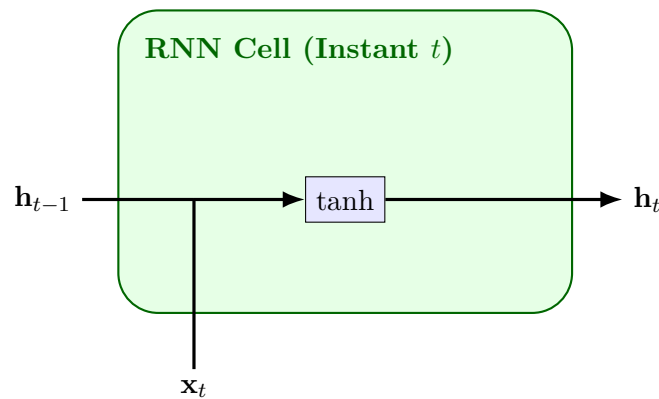


FIGURE 7 – Architecture d’une cellule RNN simple (Amnésie temporelle).

La solution : le LSTM

Pour contourner la disparition du gradient (vanishing gradient), [1] ont créé le **LSTM** (Long Short-Term Memory).

L’innovation majeure du LSTM est de ne plus confier toute la mémoire au seul vecteur $\mathbf{h}_t$. Le modèle sépare la gestion de l’information en deux flux distincts :

1. Le **Cell State** ($\mathbf{c}_t$) : C’est la mémoire long terme. Elle agit comme un *conveyor belt* (tapis roulant) qui traverse le temps en ne subissant que des opérations linéaires simples.
2. Le **Hidden State** ($\mathbf{h}_t$) : C’est la mémoire court terme. Il est calculé à partir du *cell state* mais reste influencé par l’entrée immédiate $\mathbf{x}_t$.

L’intuition mathématique : La mise à jour du *cell state* repose sur une **addition**, et non plus sur une multiplication matricielle répétée :

$$\mathbf{c}_t = \underbrace{\mathbf{f}_t \odot \mathbf{c}_{t-1}}_{\text{Oubli sélectif}} + \underbrace{\mathbf{i}_t \odot \tilde{\mathbf{c}}_t}_{\text{Ajout d'information}} \quad (12)$$

où l’opérateur $\odot$ est le **produit de Hadamard** qui multiplie les vecteurs composante par composante : $(A \odot B)_i = A_i \times B_i$.

Ici, $\mathbf{x}_t$ et $\mathbf{h}_{t-1}$ interviennent pour piloter les **gates** (portes) :

1. Le **Forget Gate** ($\mathbf{f}_t$) décide quelle partie de l’histoire ($\mathbf{c}_{t-1}$) on jette (ex : oublier un vieux prix datant d’avant une crise majeure).
2. L’ **Input Gate** ($\mathbf{i}_t$) décide quelle nouvelle information extraite de $\mathbf{x}_t$ mérite d’intégrer la mémoire long terme.

Pourquoi ça résout le problème ?

Contrairement au RNN où le gradient devait traverser des multiplications successives, le gradient ici voyage via l’opération d’addition ($\oplus$). Mathématiquement, la dérivée d’une addition est 1 : le signal ne s’écrase plus, permettant au modèle de relier des événements très éloignés dans le temps.

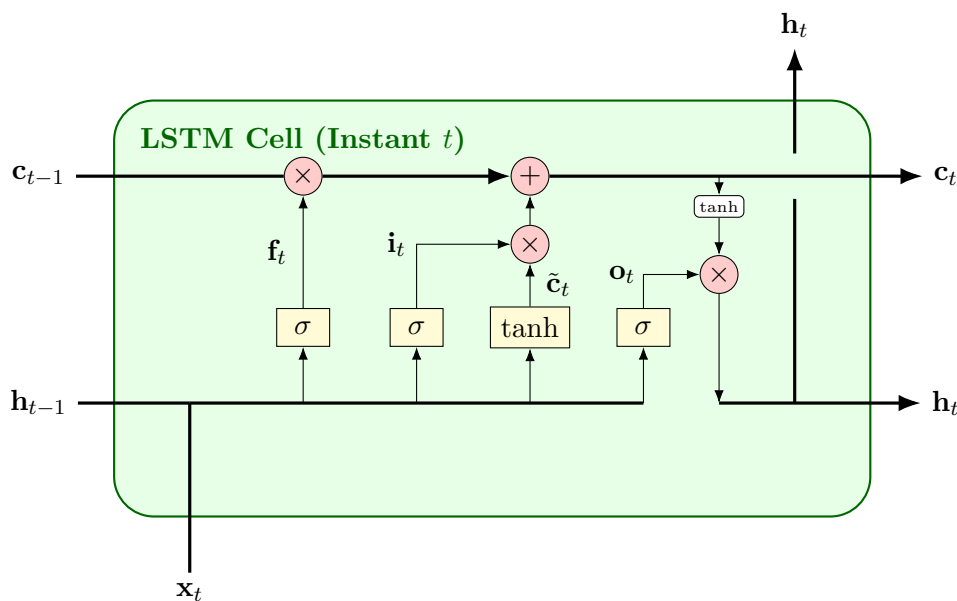


FIGURE 8 – Architecture interne d'une cellule LSTM et flux de données.

Variable	Nom technique	Rôle concret
f_t	<i>Forget Gate</i>	Décide ce qu'on efface de la mémoire long terme.
i_t	<i>Input Gate</i>	Décide quelle nouvelle info de x_t mérite d'entrer dans la mémoire long terme.
$\tilde{c}_t$	<i>Candidate</i>	La nouvelle information brute qu'on propose d'ajouter.
o_t	<i>Output Gate</i>	Le filtre qui décide quoi extraire de la mémoire long terme pour la sortie finale.
h_t	<i>Hidden State</i>	La version filtrée de la mémoire, utilisée pour la prédiction.

TABLE 1 – Rôle des variables internes de la cellule LSTM

Voici les éléments utilisés sur ce schéma :

La sortie h_t est calculée ainsi à la toute fin du bloc :

$$h_t = o_t \odot \tanh(c_t)$$

La simplification : GRU

Le LSTM est redoutable mais lourd - il contient 3 portes (f_t , i_t , o_t) et 2 états mémoires (c_t et h_t). [3] ont cherché à alléger cette mécanique sans perdre la capacité de mémoire long-terme. C'est la naissance du **GRU (Gated Recurrent Unit)**.

L'intuition mathématique : Le GRU supprime le *cell state* (c_t) pour ne conserver qu'un unique vecteur de mémoire, le *hidden state* (h_t). Pour protéger le gradient sans multiplier les composants, il fusionne les mécanismes d'oubli et d'entrée au sein d'une unique **porte de mise à jour** (*update gate*, notée z_t).

Cette architecture repose sur une **interpolation linéaire** (une moyenne pondérée) entre le passé et le présent¹ :

$$\mathbf{h}_t = \underbrace{(1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t}_{\text{Nouvelle information}} + \underbrace{\mathbf{z}_t \odot \mathbf{h}_{t-1}}_{\text{Mémoire conservée}} \quad (13)$$

où Le vecteur $\mathbf{z}_t$ agit comme un curseur : si $\mathbf{z}_t$ est proche de 1, le réseau privilégie la conservation du passé ($\mathbf{h}_{t-1}$). S'il est proche de 0, il accorde plus d'importance au candidat actuel ($\tilde{\mathbf{h}}_t$).

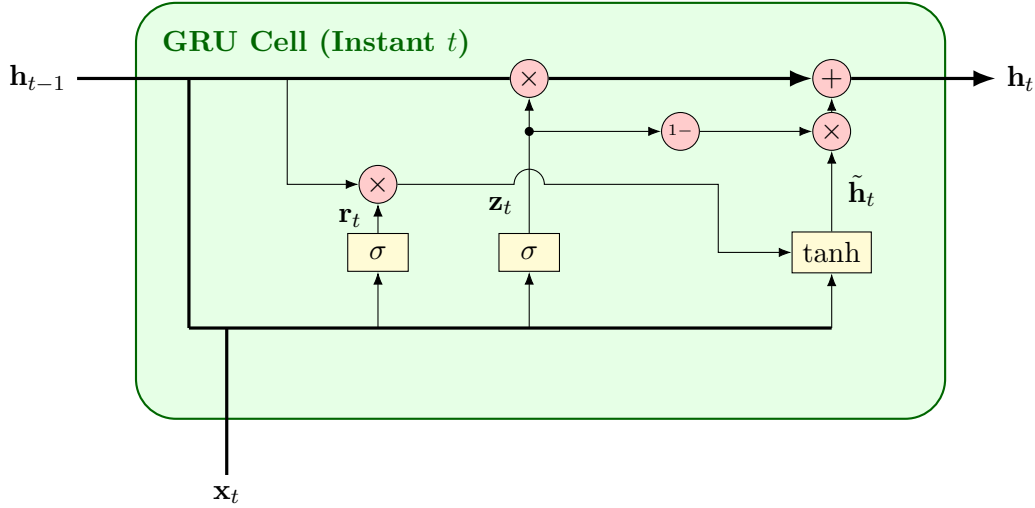


FIGURE 9 – Architecture du GRU détaillée avec les opérations matricielles sous-jacentes.

Équations internes du GRU :

$$\begin{aligned} \mathbf{z}_t &= \sigma(W_z \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_z) \\ \mathbf{r}_t &= \sigma(W_r \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + b_r) \\ \tilde{\mathbf{h}}_t &= \tanh(W \cdot [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + b) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1} \end{aligned}$$

Le rôle de chacune des variables est résumé dans la 2.

Bilan comparatif : Grâce à cette fusion, le GRU est structurellement plus léger. Il possède environ 25 % de paramètres en moins que le LSTM, ce qui lui permet de converger plus rapidement (environ 30 % de gain de temps) tout en offrant des performances prédictives souvent équivalentes sur les séries temporelles financières (voir abstract de [2] pour la rapidité et la conclusion de [3] pour les performances comparables).

1. Il est à noter qu'il existe deux conventions mathématiques dans la littérature pour le GRU. L'article original de Cho et al. (2014) définit l'interpolation comme $h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$. Cependant, les principaux frameworks de Deep Learning modernes (TensorFlow/Keras et PyTorch) ont inversé cette logique dans leur code source, utilisant z_t pour la rétention et $(1 - z_t)$ pour l'ajout. Mathématiquement, ces deux implémentations sont strictement équivalentes, le réseau adaptant le signe de ses poids synaptiques en conséquence. Nous utilisons ici la convention des frameworks modernes.

TABLE 2 – Rôle des variables internes de la cellule GRU

Variable	Nom technique	Rôle concret
$\mathbf{z}_t$	Update Gate	Décide l'équilibre entre la conservation du passé (la mémoire) et l'intégration de la nouvelle information. Elle remplace la combinaison <i>Input</i> et <i>Forget gate</i> du LSTM.
$\mathbf{r}_t$	Reset Gate	Décide quelle proportion du passé on "oublie" avant de calculer le nouveau candidat. Elle permet d'ignorer le passé si celui-ci n'est plus pertinent pour l'avenir.
$\tilde{\mathbf{h}}_t$	Candidate	La nouvelle information brute qu'on propose d'ajouter pour mettre à jour l'état du système.
$\mathbf{h}_t$	Hidden State	L'unique vecteur de mémoire. Il est la version filtrée (mise à jour) du passé, utilisée à la fois pour la prédiction et transmise au pas de temps suivant.

Le Changement de Paradigme : Les TCN (Temporal Convolutional Networks)

Malgré leurs performances, les architectures récurrentes telles que le LSTM ou le GRU souffrent d'une limite structurelle majeure : leur nature **séquentielle**. Le calcul de l'état à l'instant t impose la résolution préalable de l'état $t - 1$, interdisant de fait une parallélisation massive sur les processeurs graphiques (GPU). Pour lever ce verrou, [4] ont adapté les réseaux convolutifs aux séries temporelles, donnant naissance aux **TCN**.

Cette approche repose sur deux piliers techniques fondamentaux :

- **La Causalité** : Afin de respecter la flèche du temps et d'éviter toute « fuite » d'information du futur (*look-ahead bias*), les filtres sont décalés. La prédiction à l'instant t n'est ainsi calculée qu'à partir des valeurs $x_t, x_{t-1}, \dots$ situées à sa gauche sur la frise chronologique.
- **La Dilatation** : Pour capturer des dépendances à très long terme sans multiplier inutilement le nombre de couches, le TCN utilise des **convolutions dilatées** (voir l'encadré qui suit). En augmentant l'écart entre les points d'entrée du filtre (un pas sur deux, puis un sur quatre, etc.), le modèle élargit son champ de vision de manière exponentielle tout en restant frugal en paramètres.

L'opération mathématique d'une convolution dilatée de facteur d sur une séquence X , utilisant un filtre F de taille K , est définie par :

$$(F *_d X)(t) = \sum_{k=0}^{K-1} \mathbf{w}_k \cdot \mathbf{x}_{t-d \cdot k} \quad (14)$$

Rappel — Qu'est-ce qu'une convolution ?

Une convolution est une opération qui fait **glisser un filtre** (aussi appelé *kernel*) sur une séquence pour en extraire des motifs locaux. Intuitivement, c'est ce que fait l'œil humain quand il repère une figure chartiste : il cherche une forme caractéristique (ex : une épaule-tête-épaule) à différents endroits de la courbe.

Formellement, pour une séquence $X = (x_1, x_2, \dots, x_T)$ et un filtre $F = (w_0, w_1, \dots, w_{K-1})$ de taille K , la convolution **standard** en t est :

$$(F * X)(t) = \sum_{k=0}^{K-1} w_k \cdot x_{t-k}$$

C'est simplement une **moyenne pondérée** de K valeurs consécutives. Le filtre apprend quels poids w_k donner à chaque décalage.

La convolution dilatée introduit un écart (la *dilatation* d) entre les points lus par le filtre : au lieu de lire x_t, x_{t-1}, x_{t-2} , elle lit x_t, x_{t-d}, x_{t-2d} . Avec $d = 1, 2, 4, 8, \dots$, le champ de vision s'élargit **exponentiellement** en profondeur sans augmenter le nombre de paramètres.

Grâce à ce traitement « spatialisé » du temps, le TCN s'affranchit des problèmes d'instabilité du gradient (car beaucoup moins de multiplications matricielles sont enchaînées). De plus, contrairement au RNN où le calcul de h_t impose d'avoir résolu h_{t-1} , chaque sortie y_t d'un TCN ne dépend **que des entrées** $x_t, x_{t-d}, x_{t-2d}, \dots$ — des valeurs toutes connues dès le début. L'intégralité des sorties peut donc être calculée **en parallèle**, réduisant drastiquement les temps d'entraînement par rapport aux modèles récurrents.

Analyse de l'Architecture TCN 10 Voici les trois concepts clés illustrés par ce schéma :

- **La “Convolution Causale” (Le respect du temps)** : Les flèches ne vont que du bas (passé) vers le haut (futur). Dans une convolution causale, le modèle ne peut pas “tricher” : pour prédire la valeur à l'instant t , il n'a le droit de regarder que les données des instants $t, t-1, t-2$, etc. Il ne connaît jamais le futur. C'est essentiel car en pratique nous n'utilisons que les données passées pour prédire les cours boursiers futurs.
- **La “Dilation”** : C'est le point le plus important du schéma, noté par la valeur d à gauche :
 - $d = 1$ (Couche 1) : Le modèle regarde des points collés les uns aux autres (x_1 et x_2).
 - $d = 2$ (Couche 2) : Le modèle saute un point sur deux.
 - $d = 4$ (Couche 3) : Le modèle saute trois points.

Pourquoi faire ça ? Cela permet au neurone de sortie (y_t) d'avoir une vision globale de toute la séquence (de x_1 à x_8) avec seulement 3 couches de calcul. Sans dilatation, il aurait fallu 7 ou 8 couches pour “voir” aussi loin.

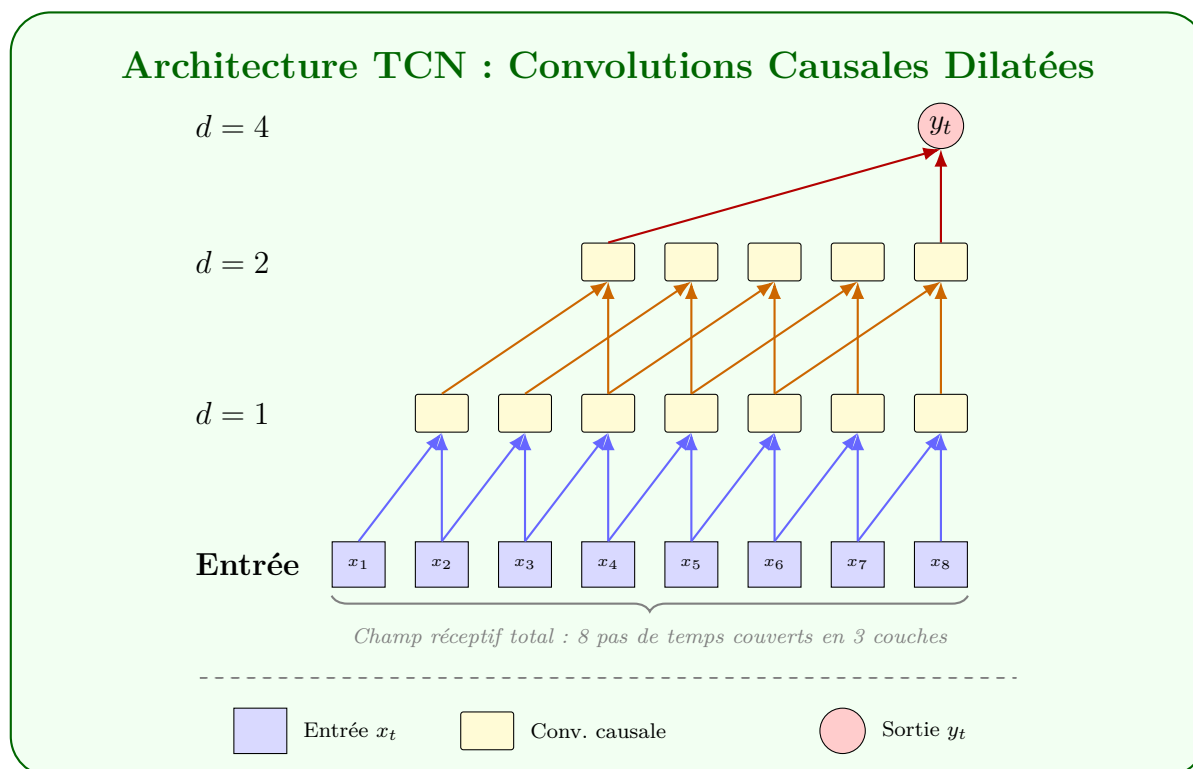


FIGURE 10 – Architecture TCN : Convolutions causales dilatées montrant le champ réceptif.

- **Le “Champ Réceptif”** : C’est l’annotation avec l’accolade en bas du schéma. Grâce à l’astuce de la dilatation ($d = 1, 2, 4$), le point de sortie y_t est connecté, par ricochet, à tous les points d’entrée de x_1 à x_8 . On dit que le champ réceptif est de 8. Le modèle a une “mémoire” de 8 pas de temps, tout en restant très léger et rapide à entraîner.

En résumé : Ce schéma montre comment une architecture TCN parvient à capter des dépendances à long terme de manière très efficace grâce à des connexions qui s’écartent de plus en plus au fur et à mesure qu’on monte dans les couches.

4. Sélection du modèle optimal

Avant de passer à la comparaison de modèles, il est essentiel de présenter le process de traitement des données ainsi que l'architecture générale du code de prédiction.

4.1. Architecture du Pipeline de Prédiction par LSTM

Ce pipeline décrit le cycle complet du traitement des données boursières, de la transformation en séquences multivariées jusqu'à l'ajustement temporel final des prédictions.

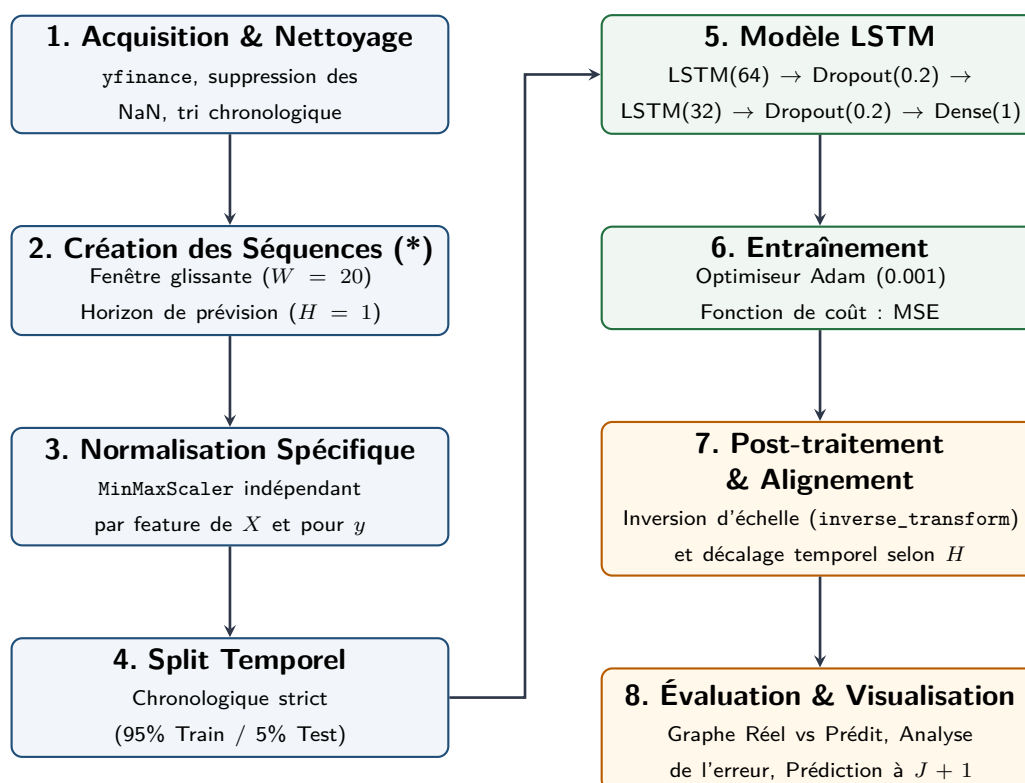


FIGURE 11 – Diagramme de flux de l'architecture de prédiction

(*) **Focus Conceptuel : Pourquoi créer des séquences ?** Dans le pipeline de préparation des données (Étape 2), la transformation en fenêtres glissantes (ou *Windowing*) est l'étape la plus critique pour un réseau récurrent : c'est grâce à cela que l'on introduit la notion de temps si différenciante avec les RNN.

Ainsi, la fonction `create_windows` transforme notre base de données 2D classique (Lignes, Features) en un tenseur 3D de dimensions (Échantillons, Pas_de_temps, Features). De ce fait, pour chaque prédiction, on donne au modèle "une phrase" complète contenant les 20 derniers jours d'historique boursier. En pratique, pour chaque instant t à prédire, le modèle va analyser une fenêtre de 20 jours précédents l'instant t et prédire la valeur. Et ainsi de suite pour tous les instants t présents dans les données d'entraînement.

Le code simplifié d'une cellule typique d'entraînement d'un modèle LSTM avec application de la fenêtre est disponible en annexe [10.1](#).

4.2. Méthodologie de comparaison itérative

Passons aux choses sérieuses ! Sur la base des recherches de la première phase, j'ai construit une boucle d'évaluation systématique sur le notebook Kaggle `choix-meilleur-modèle-bulleye-ai.ipynb`. La méthodologie arrêtée pour cette phase était la suivante :

1. Fixer les variables communes (données OHLCV via yfinance, taille de fenêtre, horizon de prédiction) ;
2. Entraîner plusieurs architectures en parallèle ;
3. Stocker la RMSE de chacune sur le jeu de test ;
4. Sélectionner les 5 meilleures architectures ;
5. Demander à Gemini de générer des variantes de ces 5 finalistes ;
6. Itérer à partir de l'étape 2.

Note : RMSE = Root Mean Square Error. C'est une mesure de l'erreur de prédiction du modèle, que l'on calcule généralement sur les données d'entraînement et sur les données de test pour évaluer sa performance.

Exemples d'architectures testées lors du premier tour :

- CNN-LSTM léger (ie avec peu de couches)
- CNN pur
- LSTM + Attention (voir Note 1 en dessous)
- GRU + CNN
- GRU bidirectionnel seul (voir Note 2 en dessous)
- GRU avec données enrichies (indicateurs techniques ajoutés)

Note 1 - Mécanisme d'Attention : Permet au modèle de ne pas traiter toutes les données passées avec la même importance. Il *pondère* les entrées : le modèle "se concentre" sur les pas de temps les plus pertinents pour prédire la sortie actuelle, améliorant la gestion des séquences très longues. Pour plus de détails, voir ce guide que je trouve intéressant : [Guide](#)

Note 2 - GRU Bidirectionnel : Contrairement à un GRU classique qui lit la séquence de t_1 à t_n , la version bidirectionnelle possède deux couches parallèles :

1. Une lecture **chronologique** (passé $\rightarrow$ présent).
2. Une lecture **antichronologique** (futur $\rightarrow$ passé).

Cela permet au réseau d'avoir un contexte complet (ce qui précède et ce qui suit) pour chaque point de donnée.

4.3. Benchmark Exhaustif des Architectures Deep Learning

Afin de déterminer la structure de modèle la plus adaptée à notre série temporelle, j'ai donc implémenté un banc d'essai évaluant 20 architectures différentes. Le code [10.2](#) teste des réseaux denses classiques, des réseaux convolutifs (CNN) pour l'extraction de caractéristiques locales, des réseaux récurrents (LSTM, GRU, BiLSTM) pour la mémoire temporelle, ainsi que des architectures hybrides. Je n'ai laissé que 5 modèles dans ce script pour faciliter la lisibilité.

Justification de l'approche Benchmark

L'évaluation comparative de 20 architectures n'est pas fortuite. Elle répond à plusieurs enjeux critiques de la modélisation financière :

- **Le No Free Lunch Theorem** : En Machine Learning, aucune architecture n'est universellement supérieure. Autrement dit, la volatilité d'une crypto-monnaie comme le Bitcoin peut être mieux captée par un réseau **BiLSTM** (qui lit la séquence dans les deux sens), tandis qu'une action très stable pourrait se contenter d'un modèle Dense bien régularisé.
- **La puissance des modèles Hybrides (CNN + LSTM/GRU)** : Le réseau convolutif (Conv1D) excelle dans l'extraction de "motifs" locaux bruts (ex : un marteau ou un doji dans les chandeliers japonais). En plaçant un CNN avant un LSTM, le CNN agit comme un filtre d'extraction de caractéristiques spatiales, et le LSTM s'occupe de modéliser la dynamique temporelle de ces caractéristiques filtrées.

4.4. Résultats

Phase 1 : Exploration globale (le benchmark large)

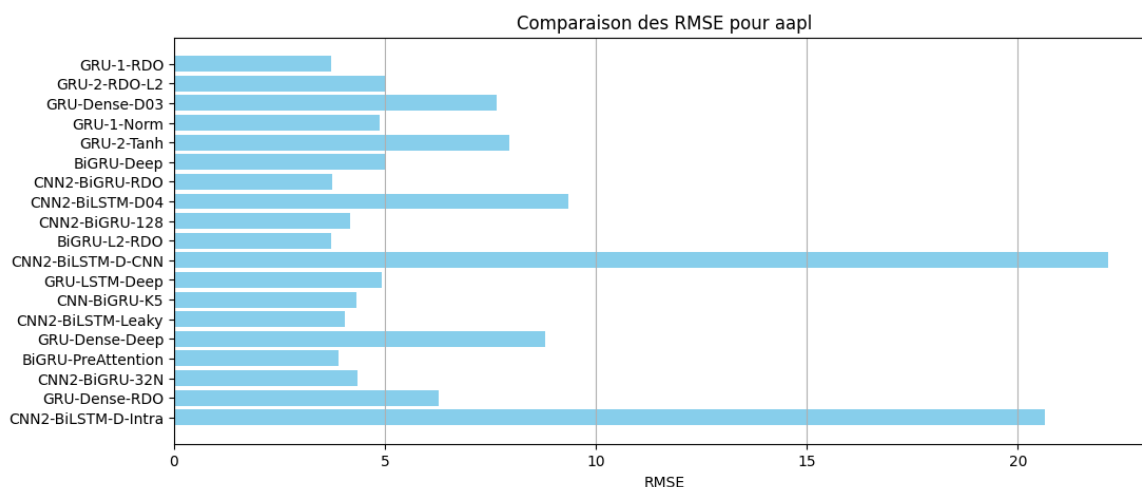


FIGURE 12 – Évaluation globale des architectures (RMSE) sur l'actif AAPL

Analyse : Comme le montre la Figure 12, le théorème du *No Free Lunch* s'applique parfaitement. Certaines architectures très profondes ou mal régularisées (comme le CNN2-BiLSTM-D-CNN ou CNN2-BiLSTM-D-Intra) explosent avec un RMSE supérieur à 20, prouvant un surapprentissage massif ou une incapacité à converger. À l'inverse, les modèles basés sur des GRU fortement régularisés par Dropout (comme le BiGRU-L2-RDO ou le GRU-1-RDO) se détachent nettement avec un RMSE sous la barre des 4.5.

Régularisation : En apprentissage profond, la régularisation regroupe un ensemble de techniques visant à réduire l'erreur de généralisation d'un modèle (ses performances sur de nouvelles données) sans pour autant diminuer son erreur d'entraînement.

Dans les faits, un modèle très complexe (comme les variantes de CNN-BiLSTM mentionnées ci-dessus) possède une grande "capacité d'apprentissage". Face à un problème difficile, il va naturellement chercher à mémoriser le bruit et les spécificités du jeu d'entraînement, ce qui mène au surapprentissage (overfitting).

La régularisation agit comme une contrainte volontaire imposée au modèle pour le forcer à la simplicité :

- Le Dropout (et Recurrent Dropout) : En masquant aléatoirement des neurones à chaque itération, on empêche les co-adaptations complexes et fragiles. Le réseau est contraint de trouver des chemins alternatifs et robustes pour faire sa prédiction.
- La pénalité L2 (Ridge) : Elle ajoute un coût mathématique à la fonction de perte proportionnel au carré de la valeur des poids. Cela force le modèle à maintenir des poids faibles et homogènes, évitant qu'une seule variable n'impacte de manière disproportionnée le résultat.

Phase 2 : Affinement avec mécanismes d'attention et TCN

Suite aux résultats de la Phase 1, nous avons écarté les modèles instables pour nous concentrer sur le gagnant temporaire (BiGRU-L2-RDO) et l'avons mis en compétition avec des architectures de pointe intégrant des réseaux convolutifs temporels (TCN) et des mécanismes d'Attention.

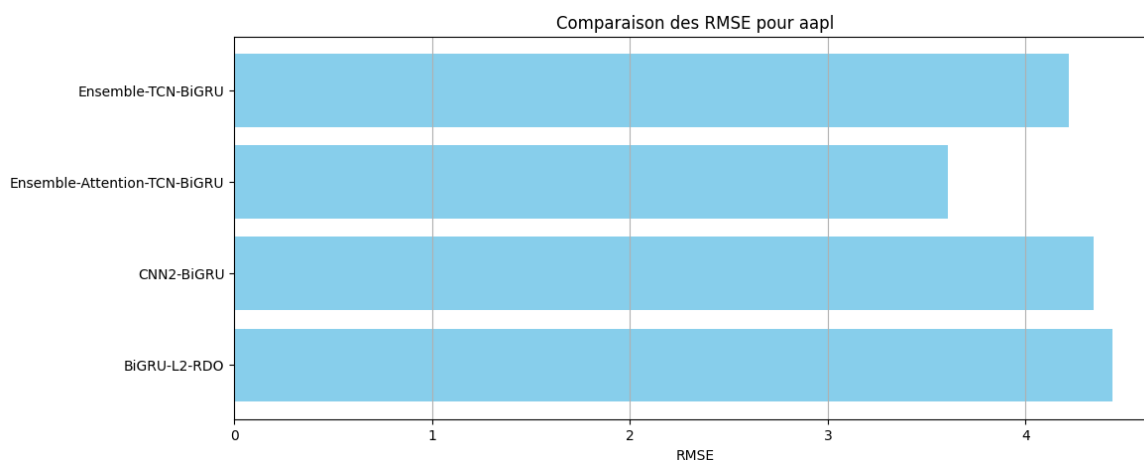


FIGURE 13 – Comparaison ciblée avec des modèles avancés (Attention et TCN)

Analyse : L'objectif ici était de voir si aider le réseau à "se concentrer" sur certains jours

spécifiques de la fenêtre de 20 jours améliorerait la prédiction. La Figure 13 révèle que le modèle **Ensemble-Attention-TCN-BiGRU** prend un léger avantage. L'architecture ensembliste (qui combine les forces de l'extraction de motifs du TCN et de la mémoire du BiGRU) abaisse le RMSE autour de 3.6, surpassant notre précédent modèle de référence.

Phase 3 : Test de robustesse

Un risque majeur en Deep Learning financier est qu'un modèle performe bien uniquement parce que son architecture s'accouple par hasard avec la fonction de coût (Loss) utilisée lors de l'entraînement. Pour vérifier que nos modèles ont réellement capté la structure du marché boursier, nous avons relancé l'entraînement de ce top 4 en changeant la fonction d'optimisation (par exemple en passant de la MSE à la MAE ou Huber Loss).

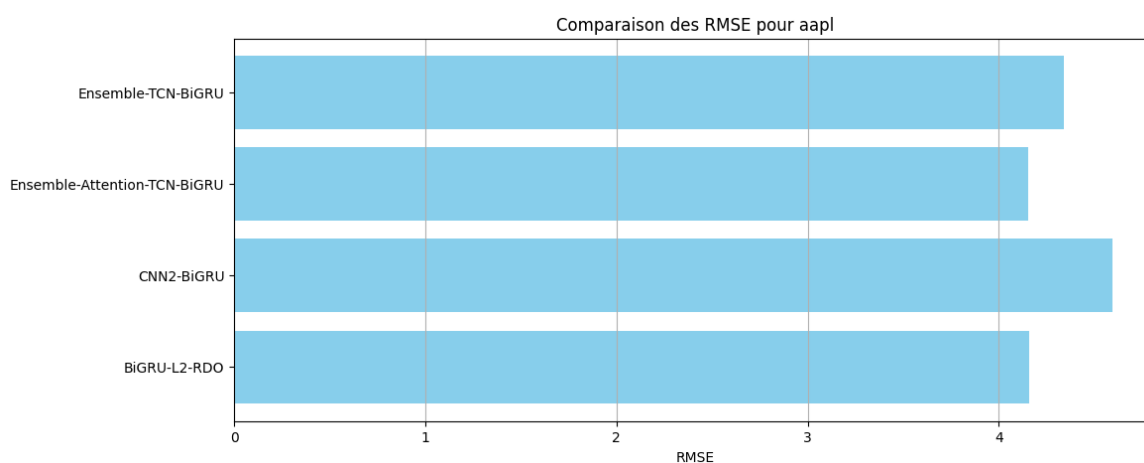


FIGURE 14 – Test de robustesse : RMSE après changement de la fonction de perte

Analyse : La Figure 14 est cruciale. Elle montre que la hiérarchie des modèles reste globalement la même. Le maintien d'un RMSE très proche pour ces architectures prouve une **robustesse structurelle**. Les modèles n'ont pas fait d'*overfitting* sur une métrique mathématique ; ils ont véritablement modélisé le signal financier sous-jacent.

Phase 4 : Dernière comparaison et verdict final

Pour l'étape finale, nous avons isolé les concepts les plus prometteurs (les Ensembles et les TCN) pour effectuer des réglages fins et des hybridations pondérées, afin de couronner l'architecture absolue pour le ticker AAPL.

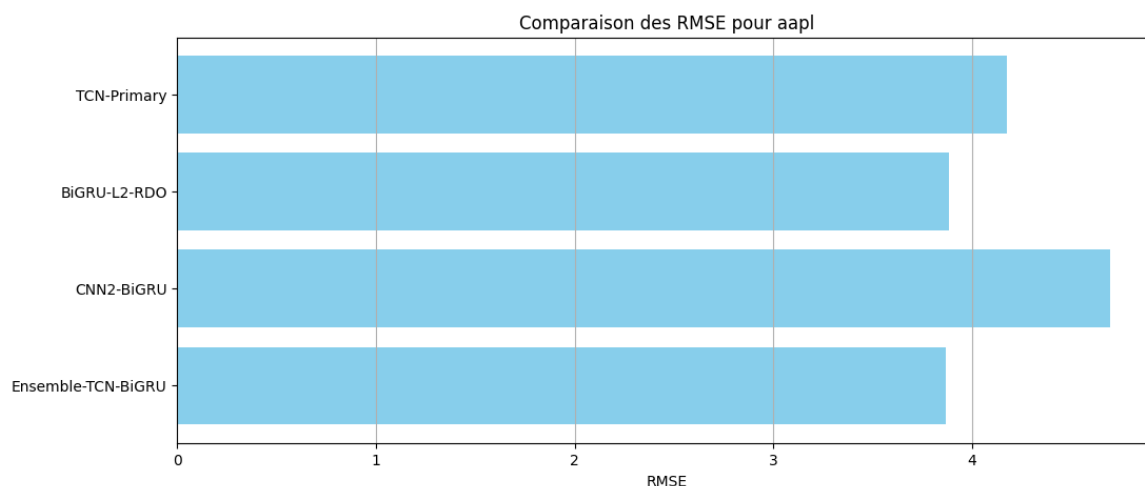


FIGURE 15 – Comparaison fine : BiGRU vs Ensembles vs TCN-Primary

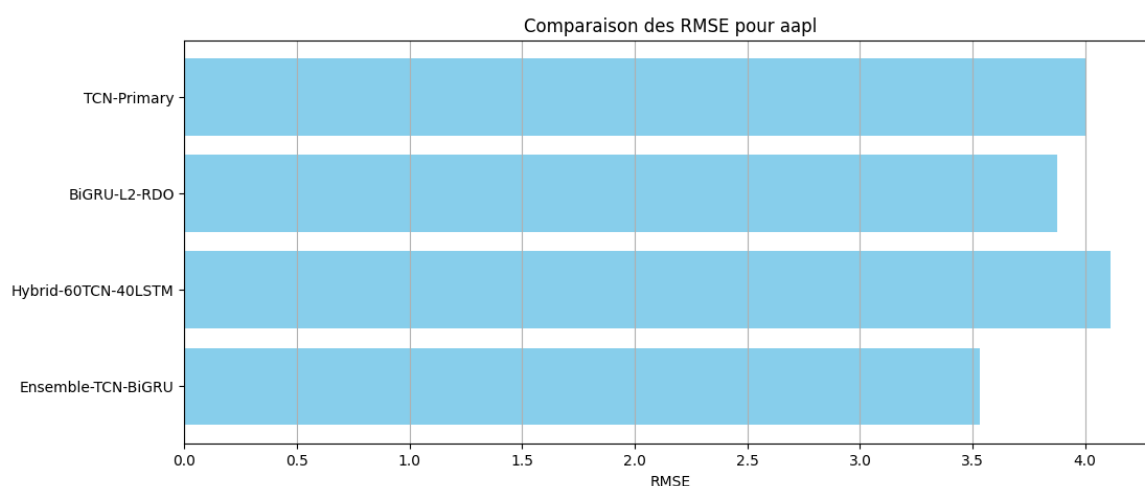


FIGURE 16 – Comparaison fine finale incluant des pondérations hybrides (60%/40%)

Conclusion de l'étude : Les Figures 15 et 16 illustrent notre ultime série de tests. Nous avons confronté notre ligne de base solide (BiGRU-L2-RDO) à un réseau convolutif temporel pur (TCN-Primary) et à des combinaisons hybrides (ex : Hybrid-60TCN-40LSTM).

Au terme de ce processus itératif, le modèle **Ensemble-TCN-BiGRU** s'impose comme le gagnant définitif. En combinant un réseau de convolutions dilaté (TCN, excellent pour repérer des motifs multi-échelles) avec un réseau récurrent bidirectionnel (BiGRU, parfait pour la dynamique séquentielle du prix), l'approche ensembliste permet d'annuler les biais individuels de chaque sous-modèle. Elle atteint le RMSE le plus bas de toute notre étude (environ 3.5), offrant une prédiction hautement résiliente au bruit boursier.

4.5. L'architecture gagnante : Attention + TCN + BiGRU

À l'issue de plusieurs cycles d'itération, l'architecture suivante s'est imposée comme la plus performante :

```

1 def create_attention_tcn_gru():
2     inputs = Input(shape=(window_size, num_features))
3
4     # Branche 1 : TCN avec convolutions dilatees
5     tcn_out = TCN(nb_filters=64, kernel_size=3,
6                   dilations=[1, 2, 4],
7                   return_sequences=True,
8                   dropout_rate=0.1)(inputs)
9
10    # Branche 2 : GRU Bidirectionnel
11    gru_out = Bidirectional(GRU(64, return_sequences=True))(
12        inputs)
13
14    # Mecanisme d'Attention
15    # L'attention compare les deux branches pour extraire les
16    # caractéristiques saillantes
17    query = Dense(128)(tcn_out)
18    value = Dense(128)(gru_out)
19    attn_out = Attention()([query, value])
20
21    avg_pool = GlobalAveragePooling1D()(attn_out)
22    dense = Dense(32, activation='leaky_relu')(avg_pool)
23    dense = Dropout(0.2)(dense)
24    output = Dense(1)(dense)
25
26    return Model(inputs=inputs, outputs=output)

```

Listing 1 – Architecture Attention + TCN + BiGRU

L'intérêt de cette combinaison réside dans la complémentarité des trois composantes :

- Le **TCN** capte les motifs de forme à court terme ;
- Le **BiGRU** modélise la tendance lourde dans les deux directions temporelles ;
- L'**Attention** permet au modèle de se concentrer sur les pics de volume précédant les grands mouvements.

4.6. Biais critique découvert durant cette phase

Biais de normalisation :

Dans tous les essais initiaux, le scaler `MinMaxScaler` était ajusté (*fit*) sur l'**ensemble des données**, y compris le jeu de test. En pratique, cela signifie que le modèle disposait d'une connaissance implicite des valeurs futures au moment de la normalisation du passé.

Conséquence : les courbes de prédiction semblaient convaincantes, mais elles étaient artificiellement "trop belles".

Enseignement : ce biais est intrinsèquement lié au travail sur des valeurs de prix absolus, et non sur des variations. Il a fortement motivé le pivot évoqué dans la partie suivante.

5. Évaluation des limites prédictives

5.1. Démarche intellectuelle et objectif du test

Après avoir identifié notre architecture optimale (**Ensemble Attention-TCN-BiGRU**) pour la prédiction à un jour ($H = 1$), une question fondamentale se pose : le modèle a-t-il réellement appris à anticiper les mouvements futurs du marché, ou se contente-t-il de se calquer sur le prix de la veille ?

Pour répondre à cette question, j'ai utilisé ce même modèle en décalant le cible temporelle (*forecast_horizon*) à 1, 3, 7, 10 et 14 jours. Cette partie est effectuée dans *robustesse_meilleur_modèle.ipynb*

Pour cela, j'ai comparé deux approches :

- **Prédiction directe** : entraîner un modèle distinct par horizon — donne de meilleurs résultats mais nécessite d'entraîner autant de modèles que d'horizon ;
- **Prédiction récursive** : réinjecter les prédictions précédentes comme données d'entrée pour les fenêtres suivantes — plus simple mais les erreurs s'accumulent rapidement.

La deuxième méthode s'est vite avérée moins efficace car elle accumule les erreurs du modèle (chaque prédiction du jour suivant s'effectue avec des données issues du modèle et donc avec une erreur par rapport aux valeurs réelles - futures - du marché).

5.2. Visualisation de la dégradation des performances

Les graphiques suivants illustrent la superposition du prix réel (en bleu) et des prédictions (en orange pointillé) pour des horizons temporels croissants.

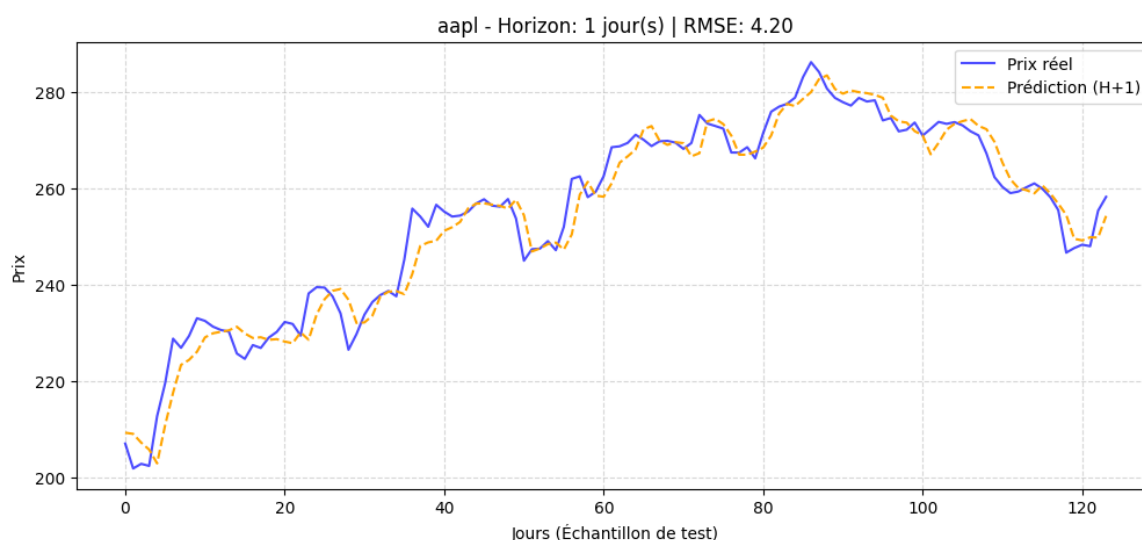


FIGURE 17 – Horizon 1 jour ($H+1$). RMSE : 4.20. La prédiction suit presque parfaitement la réalité.

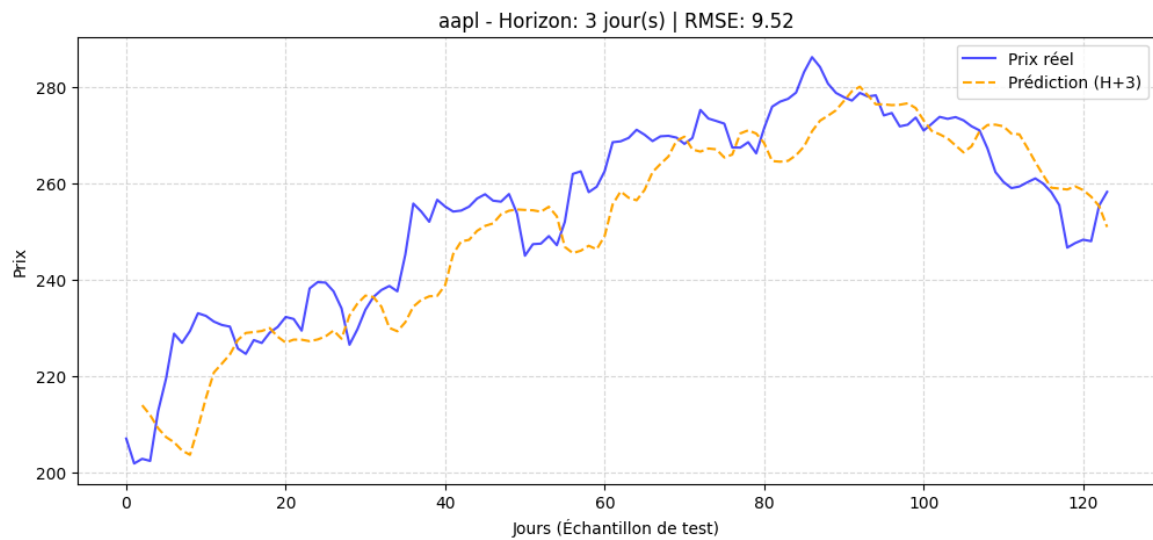


FIGURE 18 – Horizon 3 jours (H+3). RMSE : 9.52. L'erreur fait plus que doubler, un léger décalage visuel apparaît.

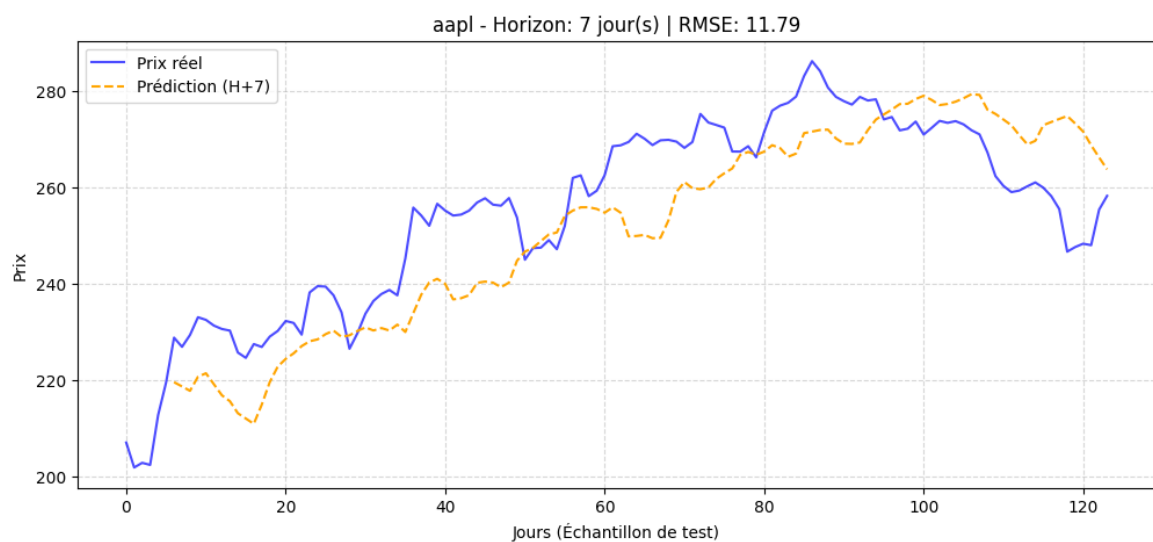


FIGURE 19 – Horizon 7 jours (H+7). RMSE : 11.79. Le modèle commence à lisser ses prédictions.

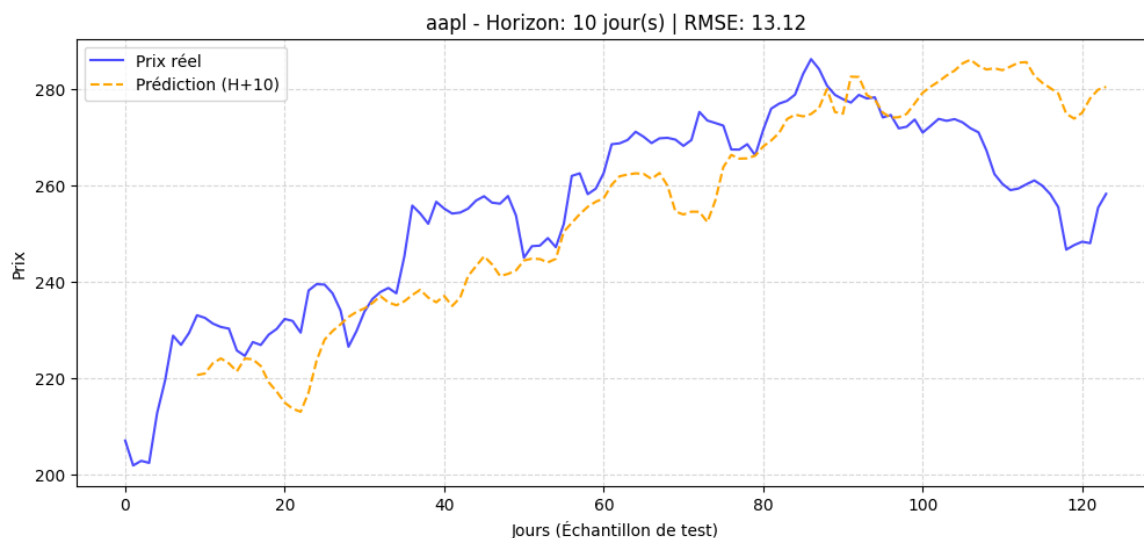


FIGURE 20 – Horizon 10 jours (H+10). RMSE : 13.12. Le décalage temporel devient flagrant.

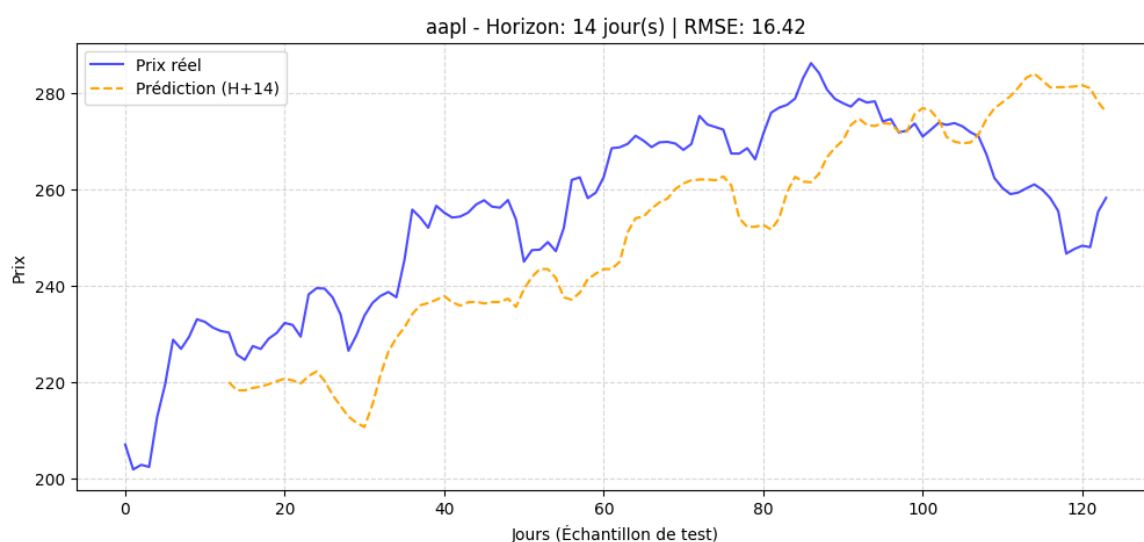


FIGURE 21 – Horizon 14 jours (H+14). RMSE : 16.42. Rupture totale : la prédiction est visuellement déphasée par rapport à la réalité.

5.3. Analyse

L'évolution de la RMSE (qui passe de 4.20 à 16.42) et l'observation visuelle des courbes mettent en lumière plusieurs conclusions critiques sur la modélisation financière par Deep Learning :

- **La perte d'anticipation et l'effet de "Lag"** : Sur la Figure 17 (H+1), le modèle est très réactif. Cependant, dès la Figure 19 (H+7) et de manière flagrante sur la Figure 21 (H+14), la courbe orange ne "prédit" plus les retournements de tendance : elle les subit avec du retard. Visuellement, la prédiction ressemble à une copie du prix réel qui aurait été décalée vers la droite.

- **Le repli vers une stratégie naïve** : Mathématiquement, quand le réseau de neurones réalise que l'incertitude est trop grande pour un horizon lointain, l'optimiseur (Adam) va chercher à minimiser l'erreur (MSE) de la façon la plus sûre possible. La stratégie la plus sûre pour un algorithme face à l'inconnu est de prédire une valeur très proche du dernier prix connu (ou une moyenne lissée récente). C'est pourquoi la courbe orange devient plus "douce" et retardée.
- **Hypothèse de l'Efficiencia des Marchés (EMH)** : Cette dégradation montre que les séries temporelles boursières s'apparentent à une *marche aléatoire* sur le moyen/long terme. Les 20 jours passés (notre *window_size*) contiennent suffisamment d'autocorrélation pour estimer le prix de demain, mais l'information se dissipe totalement à 14 jours. Des facteurs externes (nouvelles macroéconomiques, annonces de l'entreprise) non fournis au modèle prennent alors le relais pour dicter le prix.

Conclusion : Tenter d'utiliser le modèle hybride pour des horizons supérieurs à une semaine sans y intégrer de données exogènes (Analyse du sentiment (NLP), données fondamentales de l'entreprise) s'avère inefficace, celui-ci se transformant de facto en un simple indicateur de moyenne mobile retardé.

6. Modèles de classification et enrichissement des données

6.1. Pivot stratégique : de la régression vers la classification

Le 3 janvier 2026, une réflexion de fond m'a conduit à une décision importante :

Passage en classification :

Plutôt que de prédire une valeur exacte de prix (très bruitée), l'objectif devient de prédire la **direction** du mouvement. Le modèle classifiera chaque point en trois classes :

- **Classe 0** : Hausse significative ($> +2\%$) ;
- **Classe 1** : Baisse significative ($> -2\%$) ;
- **Classe 2** : Stabilité (entre -2% et $+2\%$).

Note : le 2% est ici arbitraire.

Avantages de ce paradigme :

- Élimine l'effet miroir, ie que le modèle doit choisir une direction, il ne peut plus "suivre" ;
- Plus robuste au bruit de marché car prédire un seuil est plus fiable que prédire une valeur exacte ;
- Les données sont également transformées : on passe des valeurs de prix brutes aux **pourcentages de variation**, ce qui est mathématiquement plus robuste et élimine structurellement le biais de normalisation identifié précédemment ;
- L'architecture Attention + TCN + GRU reste valide — il suffit de remplacer la couche de sortie par `Dense(3, activation='softmax')` et la perte par `categorical_crossentropy`.

* j'ai mis un peu de temps à me convaincre de ce point mais il est en fait bien réel. En réalité nous avons vu que pour un modèle de régression, la stratégie gagnante pour une prédiction à un jour est généralement de prédire le prix actuel pour le prix de demain. Que le prix monte ou baisse la RMSE sera acceptable et le modèle s'en contente.

Dans le cas d'une classification, le modèle ne peut plus raisonner ainsi car s'il garde le même prix et que ce choix ne va pas dans le bon sens, alors il sera beaucoup plus pénalisé car il se sera trompé à 100% (si le modèle a choisi la mauvaise classe, il se trompe complètement, alors que s'il vise une valeur un peu à côté en régression il ne se trompe qu'un petit peu).

Le notebook `bulleye-ai-classification.ipynb` matérialise ce pivot.

Les expériences menées m'ont permis d'explorer :

- L'impact de la réduction du nombre d'indicateurs en entrée ;
- La variation de la taille de fenêtre temporelle ;

- L'impact du changement d'horizon (de 7 à 1 jour(s));
- Un lissage de la cible : au lieu de regarder le prix dans exactement n jours, utiliser la **moyenne glissante** des jours 8, 9, 10 pour atténuer la volatilité de la cible.

Pour cette nouvelle phase d'expérimentation et comme expliqué précédemment, j'ai fondamentalement modifié mon approche. Au lieu d'essayer de deviner le prix exact (régression), j'ai transformé le problème en **Classification** via la méthode du *Triple Barrier Labeling*. L'objectif est de prédire une direction forte : Achat (BUY), Vente (SELL) ou Attente (HOLD), en fixant un seuil de variation (dans la suite 3.5%).

```

1  # Labeling Triple Barrier
2  future_change = df['Close'].shift(-HORIZON) / df['Close'] - 1
3  THRESHOLD = 0.035
4
5  def labeler(x):
6      if pd.isna(x): return None
7      if x > THRESHOLD: return 2    # BUY (UP)
8      if x < -THRESHOLD: return 0   # SELL (DOWN)
9      return 1                     # HOLD (NEUTRAL)
10
11 df['target'] = future_change.apply(labeler)
12
13 # [...] Entraînement des modeles [...]
14
15 # Filtre de Haute Confiance - ici > 50% de confiance
16 CONFIDENCE_THRESHOLD = 0.5
17 y_pred_probs = model.predict(X_test)
18 y_pred_classes = np.argmax(y_pred_probs, axis=1)
19 y_pred_max_prob = np.max(y_pred_probs, axis=1)
20
21 # On ne garde que les signaux pour lesquels le modele
22 # est confiant
23 strong_signal_mask = (y_pred_max_prob > CONFIDENCE_THRESHOLD)
    & (y_pred_classes != 1)

```

Listing 2 – Implémentation du Triple Barrier Labeling et du filtre de confiance

6.2. Impact de la réduction du nombre d'indicateurs en entrée

En réduisant l'historique de données, la hiérarchie des modèles a été complètement modifiée.

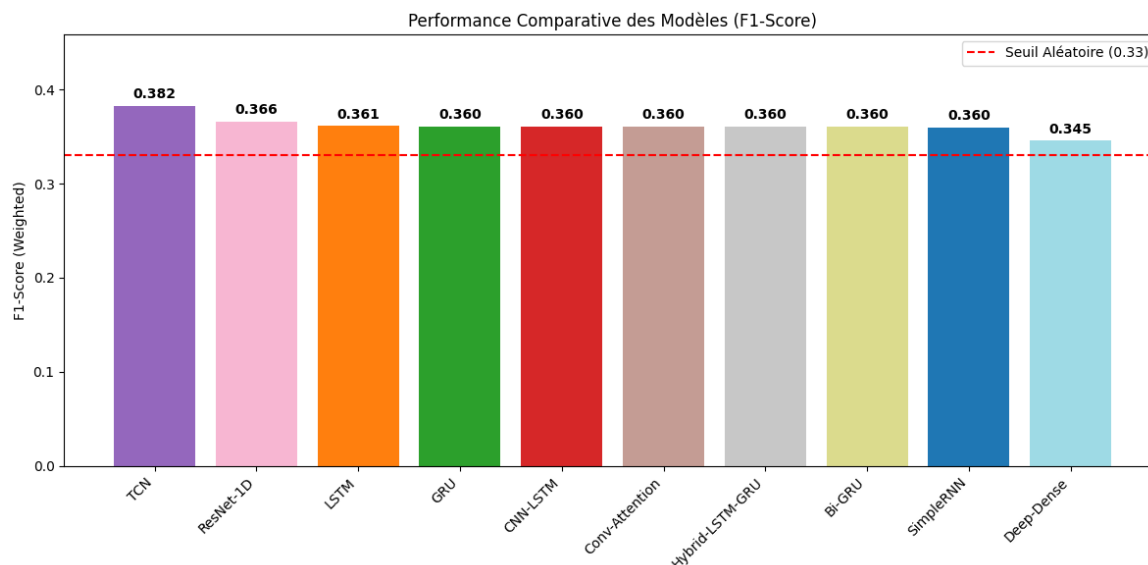


FIGURE 22 – F1-Score avec grand historique (TCN vainqueur)

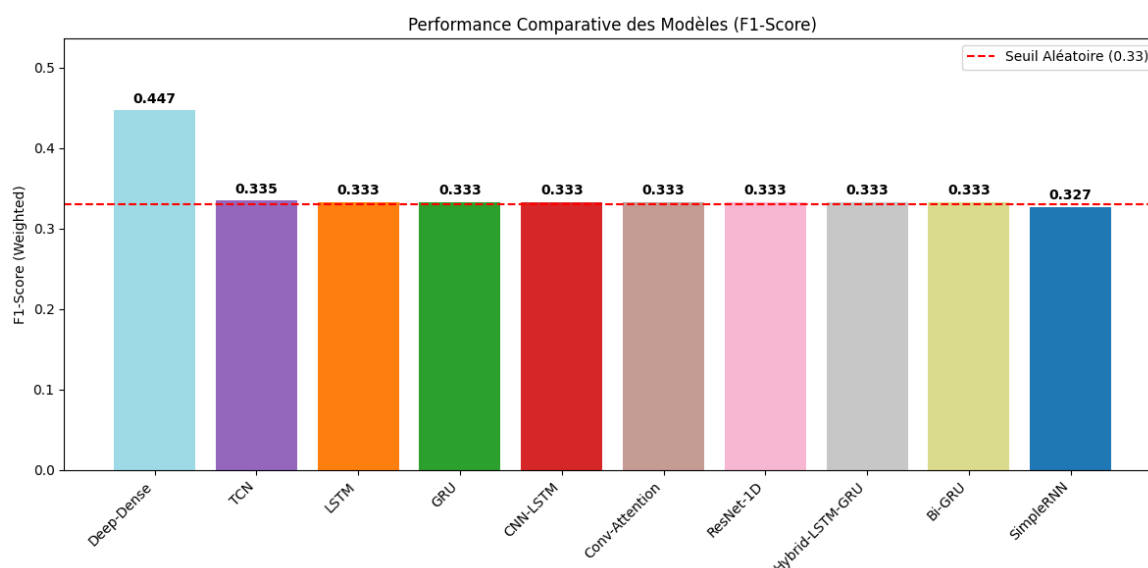
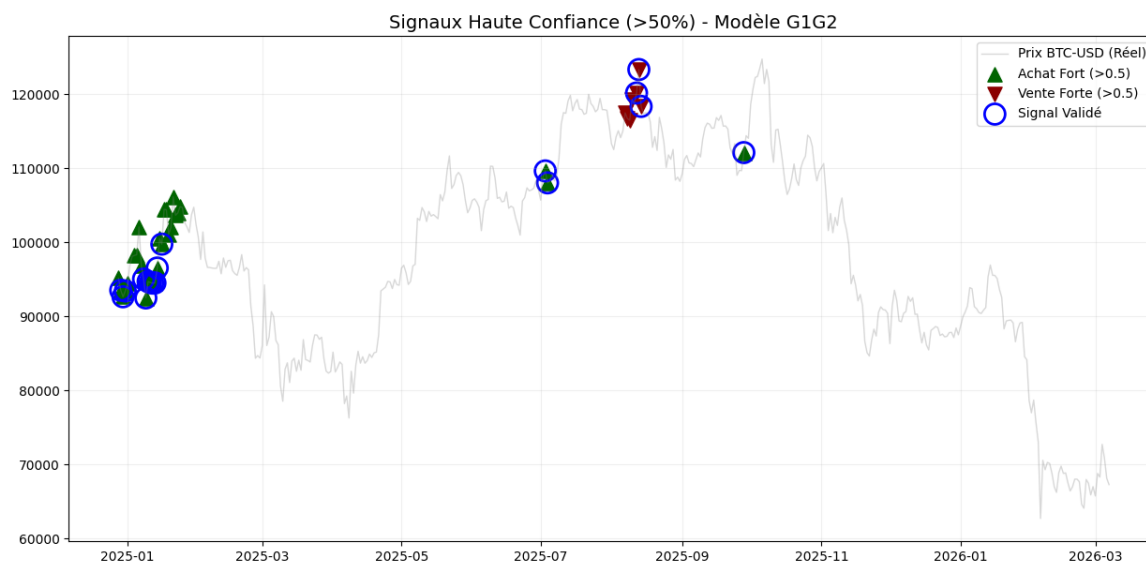
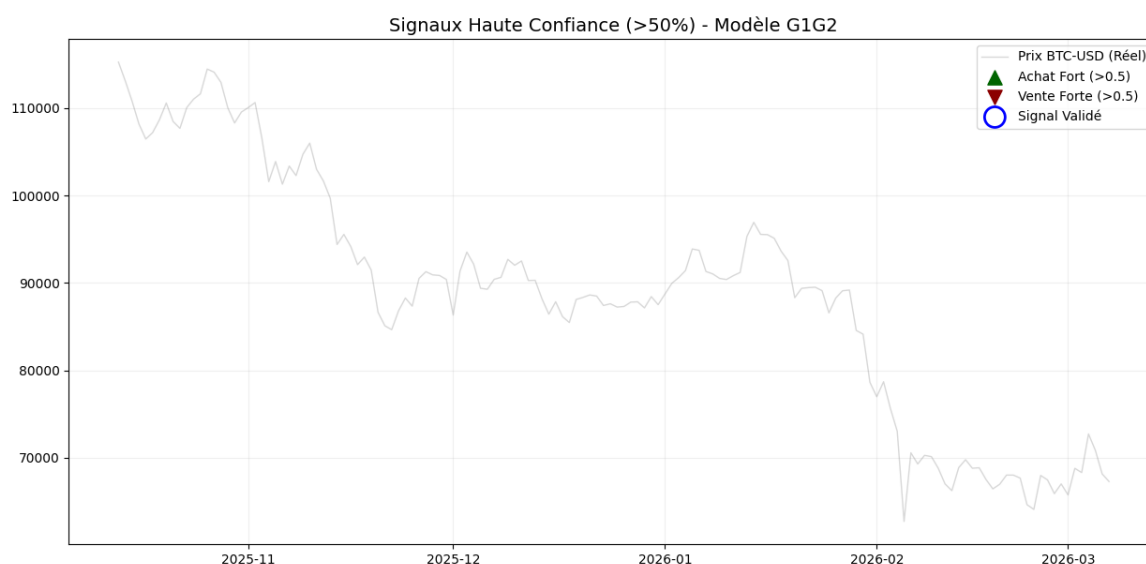


FIGURE 23 – F1-Score avec historique réduit (Dense vainqueur)

Analyse : Sur un grand historique (Figure 22), les modèles séquentiels (TCN, ResNet, LSTM) dominent car ils ont assez de données pour apprendre des dynamiques temporelles complexes. À l'inverse, en restreignant les données à 2024 (Figure 23), le modèle **Deep-Dense** prend la première place avec un F1-Score impressionnant de 0.447. Cependant, ce résultat est une anomalie statistique. Les réseaux séquentiels sont "affamés" de données et sous-performent sur de petits échantillons, tandis que le réseau Dense, plus "naïf", a simplement mémorisé les caractéristiques statiques de l'année 2024 (surapprentissage).

En réalité, un F1-score élevé ne suffit pas si le modèle n'a aucune conviction. L'ultime test consiste à n'exécuter un trade que si la probabilité de la classe prédite via la couche **Softmax** dépasse 50% - le fameux *confidence_threshold* du code 6.1.

FIGURE 24 – Signaux générés par l'architecture récurrente hybride avec filtre $> 50\%$ FIGURE 25 – Signaux générés par le modèle Deep-Dense avec filtre $> 50\%$: un vide quasi total.

Analyse et Conclusion

La comparaison entre la Figure 24 et la Figure 25 est révélatrice. Bien que le modèle Deep-Dense affiche un meilleur F1-Score global, la Figure 25 montre qu'il ne produit quasiment aucun signal lorsqu'on lui demande d'être sûr de lui à plus de 50%.

Comment expliquer ce phénomène ?

Le modèle Dense a une très mauvaise *calibration*. Pour une prédiction donnée, sa couche **Softmax** va par exemple sortir les probabilités suivantes : [BUY: 38%, HOLD: 32%, SELL: 30%]. Puisque 38% est la valeur maximale, le modèle choisit "BUY". Si

le marché monte effectivement, le F1-Score s'améliore. Mais mathématiquement, le modèle "hésite" énormément (38% est très faible).

Dès que l'on applique notre masque `strong_signal_mask` (> 0.5), le modèle Dense est éliminé, car ses probabilités ne franchissent jamais ce seuil. À l'inverse, un modèle récurrent bien entraîné sur un historique suffisant sera capable de générer des probabilités de l'ordre de 60% ou 70% lorsqu'il reconnaît une structure temporelle forte.

En trading algorithmique, un modèle moyennement précis mais fortement calibré (sûr de lui quand il a raison) est infiniment supérieur à un modèle statistiquement plus précis mais perpétuellement indécis.

7. Pipeline de classification final et résultats

7.1. De la barrière statique à la barrière dynamique

Le premier pipeline de classification (`bulleye-ai-classification.ipynb`) utilisait un seuil fixe `THRESHOLD = 0.035` sur Bitcoin. Transposé au SPY, ce seuil statique s'est révélé structurellement inadapté : sur un actif peu volatil dont la volatilité journalière oscille entre 0,5 % (marché calme, 2023–2024) et 3 % (crise COVID, 2020), un seuil fixe produit des labels dont la signification change selon le régime de marché - ce qui est finalement assez naturel.

La solution adoptée est une **barrière dynamique** proportionnelle à la volatilité réalisée :

$$\text{seuil}(t) = \sigma_{20j}(t) \times \sqrt{H} \times \lambda \quad (15)$$

où $\sigma_{20j}(t)$ est l'écart-type des rendements sur les 20 derniers jours, H l'horizon de prédiction et λ le multiplicateur de volatilité. Ce seuil s'adapte au régime courant : il est élevé en période agitée (évitant de labelliser comme signal fort un mouvement ordinaire) et bas en période calme.

```

1 def get_dynamic_labels(df, horizon, multiplier):
2     prices = df['Close'].values
3     vol     = df['returns'].rolling(20).std().values
4     labels = []
5     for i in range(len(prices) - horizon):
6         threshold = vol[i] * np.sqrt(horizon) * multiplier
7         upper = prices[i] * (1 + threshold) # barriere BUY
8         lower = prices[i] * (1 - threshold) # barriere SELL
9         label = 1 # HOLD par défaut (barriere temporelle)
10        for p in prices[i+1 : i+horizon+1]:
11            if p >= upper: label = 2; break # BUY
12            elif p <= lower: label = 0; break # SELL
13        labels.append(label)
14    return labels

```

Listing 3 – Implémentation de la triple barrière dynamique

La calibration du multiplicateur λ (= constante `VOL_MULTIPLIER` dans le notebook) a nécessité plusieurs itérations :

λ	Proportion HOLD	Problème
0,75	41 %	Barrière trop basse : les labels capturent le bruit ordinaire du SPY
1,00	59,5 %	Équilibre acceptable — configuration finale retenue
1,50	79,4 %	HOLD domine : le modèle n'apprend plus à détecter BUY/SELL

TABLE 3 – Impact du multiplicateur λ sur la distribution des classes (SPY 2018–2026, $H = 3$)

La configuration finale retenue est $\lambda = 1,0$, $H = 3$ jours, avec la distribution : **HOLD 59,5 %**, **BUY 21,9 %**, **SELL 18,6 %**.

7.2. Enrichissement des données et pipeline technique

Le pipeline final mobilise 17 features réparties en quatre catégories. Aux indicateurs techniques classiques (returns, RSI, MACD, Stochastique, BB_width, ADX, Volatilité) et aux features de structure de marché (volume_change, volume_ma_ratio, high_low_range, close_position, momentum_3/7/14) s'ajoutent deux données **macro-économiques** et une donnée de **sentiment** :

- **VIX** (normalisé) : indice de volatilité implicite du S&P 500, mesure de l'anxiété du marché ;
- **TNX** (normalisé) : taux d'intérêt 10 ans américain, principal signal macroéconomique mondial ;
- **Fear & Greed Index** : indice composite (CNN Money) agrégeant 7 signaux de sentiment, normalisé en $[0, 1]$. C'est la seule donnée partiellement causale du pipeline.

Une description détaillée de tous ces indicateurs est disponible en [10.3](#)

Deux corrections techniques critiques distinguent ce pipeline de ses prédécesseurs :

- **RobustScaler au lieu de MinMaxScaler** : Le MinMaxScaler est sensible aux valeurs extrêmes et, fitté sur l'ensemble train+test, introduisait le biais de normalisation identifié [4.6](#). Le RobustScaler (basé sur la médiane et l'écart interquartile) est insensible aux pics de volatilité. Il est fitté *uniquement* sur le train set, puis appliqué en `transform` sur le test.
- **Poids de classe** : Avec 59,5 % de HOLD, un modèle non régularisé apprend à tout prédire HOLD. Les `class_weights` pénalisent proportionnellement les erreurs sur les classes rares ($w_{\text{SELL}} \approx 1,73$, $w_{\text{BUY}} \approx 1,51$, $w_{\text{HOLD}} \approx 0,56$).

7.3. Benchmark des 9 architectures

Neuf architectures ont été comparées sur le même pipeline, entraînées avec **EarlyStopping** (patience= 10) et **ReduceLROnPlateau** (factor= 0,5). Le classement par F1-macro (voir définition ici [7.4.1](#)) et par accuracy sur les signaux à forte confiance ($> 55\%$) est présenté dans le tableau [4](#).

Modèle	F1-macro	Conf. moy.	Acc. > 55 %	Nb signaux
Bi-GRU	0,416	0,425	79,17 %	24
Attention-TCN-GRU	0,380	0,501	66,67 %	63
SimpleRNN	0,380	0,470	61,36 %	44
Deep-Dense	0,373	0,494	58,73 %	63
Hybrid-LSTM-GRU	0,372	0,433	90,00 %	10
CNN-LSTM	0,363	0,540	63,64 %	110
TCN	0,352	0,477	76,00 %	50
LSTM	0,345	0,455	78,57 %	28
Conv-Attention	0,330	0,415	82,35 %	17

TABLE 4 – Classement des 9 architectures ($\lambda = 1,0$, $H = 3$, SPY 2018–2026)

Le modèle Hybrid-LSTM-GRU affiche l'accuracy la plus élevée sur signaux forts (90 % à > 55 % de confiance), mais sur seulement 10 signaux en 269 jours de test (un échantillon trop restreint pour conclure à une robustesse statistique). L'ATG, avec 62 signaux et une progression monotone de l'accuracy avec le seuil, offre une validation plus solide.

Note : Le F1-macro est retenu comme métrique principale car il traite chaque classe (SELL, HOLD, BUY) avec un poids identique, évitant qu'un modèle prédisant systématiquement HOLD ne soit artificiellement avantagé par la distribution déséquilibrée des labels.

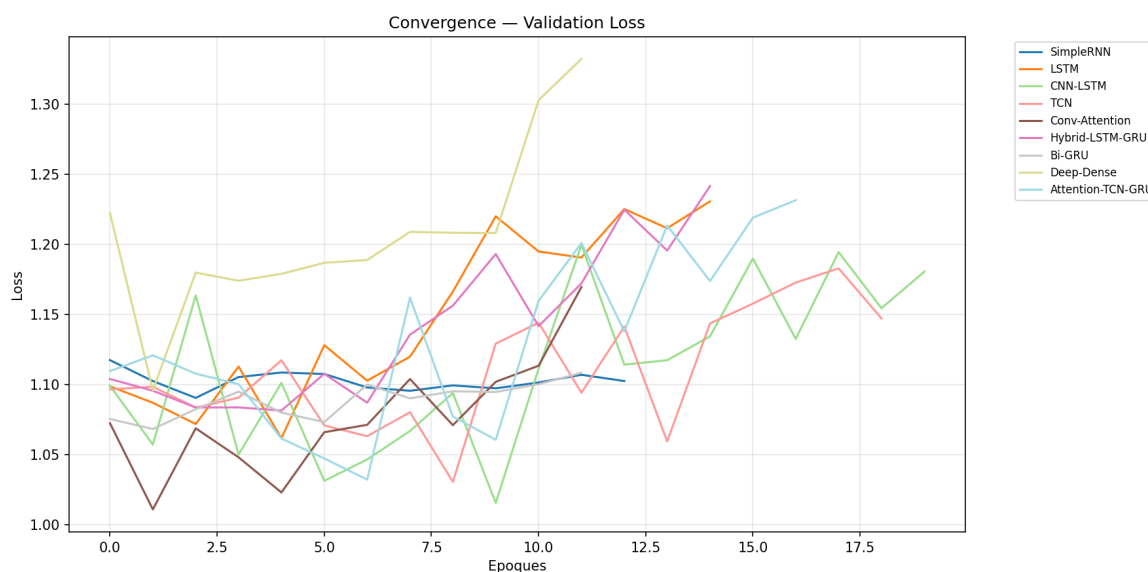


FIGURE 26 – Courbes de convergence (val_loss) des 9 architectures au cours de l'entraînement

Tous les modèles présentent une confiance moyenne inférieure à 0,55, signe que le signal est faible. Cependant, la colonne « Acc. > 55 % » révèle un phénomène crucial : lorsque l'on ne retient que les signaux sur lesquels le modèle est confiant, l'accuracy augmente fortement. Le Hybrid-LSTM-GRU atteint ainsi 90 % sur 10 signaux, et le Conv-Attention 82 % sur 17 signaux.

7.4. Justification du choix de l'architecture ATG

À la lecture du classement global (Tableau 4), il peut paraître paradoxal de sélectionner l'architecture Attention-TCN-GRU (ATG) alors qu'elle affiche un score F_1 macro inférieur à celui du modèle Deep-Dense. Pour comprendre ce choix, il est impératif de déconstruire le calcul de ce score et d'analyser la répartition réelle des erreurs via la matrice de confusion.

Définition du F_1 -Score, de la Précision et du Rappel

Le score F_1 est la moyenne harmonique de la Précision et du Rappel. Il est défini par l'équation suivante :

$$F_1 = 2 \times \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \quad (16)$$

Afin de cerner les limites de cette métrique dans un contexte financier (par exemple, lors de la prédiction d'un signal BUY), il faut redéfinir ses composantes fondamentales :

- **La Précision** : Elle mesure la proportion de prédictions correctes parmi tous les signaux déclenchés par le modèle.

$$\text{Précision} = \frac{TP}{TP + FP} = \frac{1}{1 + \frac{FP}{TP}} \quad (17)$$

Où TP désigne les Vrais Positifs (les hausses correctement anticipées) et FP les Faux Positifs (les fausses alertes). En trading algorithmique, une faible précision se traduit directement par des pertes de capital.

- **Le Rappel** : Il mesure la proportion des mouvements réels du marché que le modèle a réussi à capter.

$$\text{Rappel} = \frac{TP}{TP + FN} = \frac{1}{1 + \frac{FN}{TP}} \quad (18)$$

Où FN désigne les Faux Négatifs. En investissement, un faible rappel représente une opportunité manquée (le capital n'est pas investi, il n'y a donc pas de perte brute).

La limite structurelle du score F_1 est qu'il accorde un poids équivalent à la Précision et au Rappel. Or, dans le cadre d'une stratégie d'investissement, le coût d'un FP (perte financière) est asymétriquement supérieur au coût d'un FN (statu quo).

L'illusion du Deep-Dense

Le fait que le modèle Deep-Dense obtienne un meilleur score F_1 n'est pas le reflet d'une meilleure lecture du marché, mais une illusion statistique due au couplage de la fonction Softmax et de l'opérateur argmax.

Tous les modèles peuvent générer des TP par chance. Cependant, face au "bruit" du marché, le Deep-Dense présente une entropie très élevée (= forte incertitude) : il ne produit quasiment jamais de probabilités de confiance supérieures à 50 %. Son vecteur

de prédiction type ressemble à $\hat{y} = [0.38, 0.32, 0.30]$. Mathématiquement, l'incertitude du modèle est maximale. Néanmoins, l'opérateur argmax force une classification absolue et assigne ce point à la classe BUY (puisque 38 % est la valeur maximale).

Parce qu'il passe son temps à "deviner" avec des signaux faibles, le Deep-Dense émet un volume colossal de prédictions aléatoires. En vertu de la loi des grands nombres, il va inévitablement croiser des mouvements réels du marché et accumuler des TP par simple coïncidence. Cette inflation aléatoire des TP maintient son Rappel - et par extension son score F_1 - à un niveau élevé. Or engager du capital avec seulement 38 % de certitude garantit une espérance mathématique strictement négative.

Analyse de la Matrice de Confusion

À l'inverse, l'architecture ATG possède une distribution probabiliste **calibrée** : le réseau est capable de modéliser sa propre incertitude. L'application d'un seuil de confiance ($\tau > 0.55$) filtre drastiquement le bruit.

Le résultat important de cette partie n'est effectivement pas l'accuracy globale (qui reste modeste), mais la **progression de l'accuracy en fonction du seuil de confiance** sur le modèle ATG :

Seuil de confiance	Signaux retenus	Accuracy
> 55 %	62	61,29 %
> 60 %	42	64,29 %
> 65 %	24	79,17 %
> 70 %	22	77,27 %

TABLE 5 – Accuracy du modèle ATG selon le seuil de confiance (jeu de test, 272 jours)

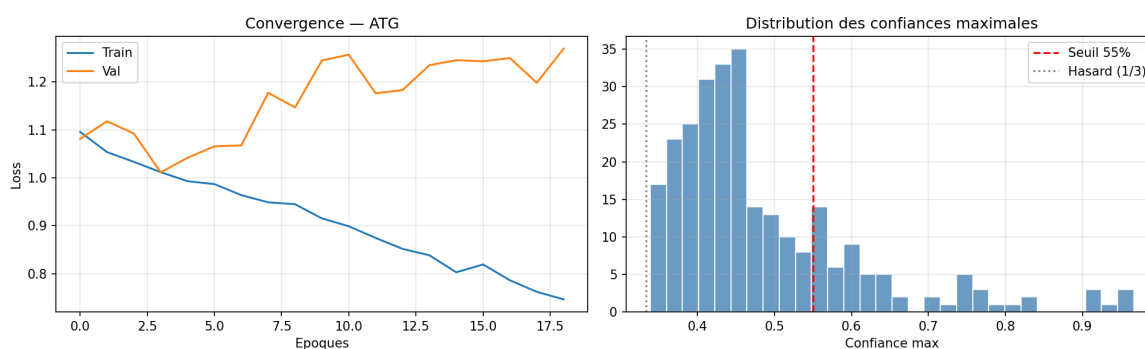


FIGURE 27 – Convergence du modèle ATG (gauche) et distribution des confiances maximales (droite)

Sur les 62 signaux retenus, la répartition des performances par classe est la suivante :

- **Classe HOLD (Stabilité)** : 35 prédictions correctes sur 42 signaux générés, soit une Précision de 83,33 %.
- **Classe BUY (Hausse)** : 2 prédictions correctes sur 6 signaux générés, soit une Précision de 33,33 %.
- **Classe SELL (Baisse)** : 1 prédiction correcte sur 14 signaux générés, soit une Précision de 7,14 %.

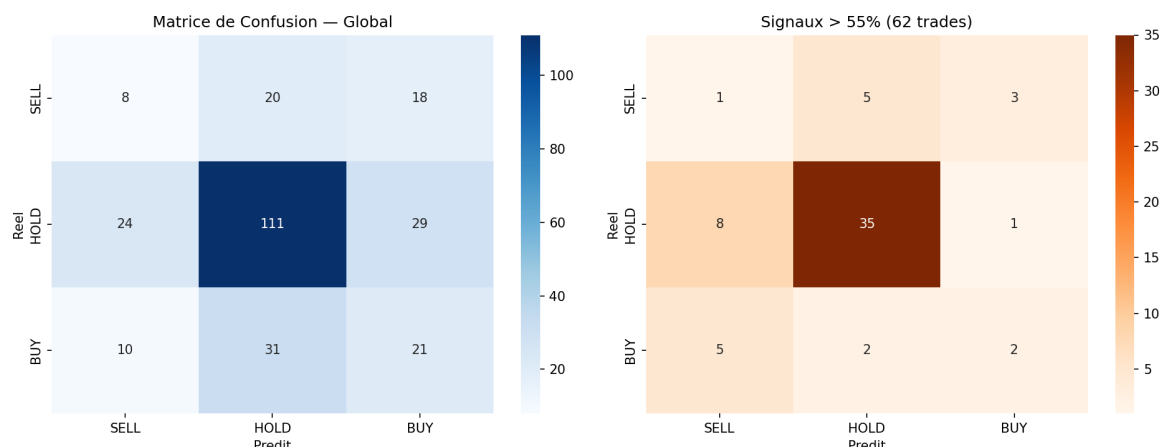


FIGURE 28 – Matrices de confusion : tous signaux (gauche) vs signaux à confiance > 55 % (droite)

Les données sont incontestables : **le modèle est structurellement incompetent pour prédire les retournements directionnels (BUY et SELL)**. Même lorsqu'il se déclare "confiant", son taux d'erreur sur ces classes reste rédhibitoire.

Toutefois, sa précision remarquable sur la classe HOLD (83,33 %) démontre que le modèle excelle à détecter **l'inertie** du marché. Le réseau a mathématiquement appris à identifier les régimes de faible volatilité où le marché n'a aucune raison directionnelle de bouger.

En conclusion, le choix de l'ATG se justifie précisément parce qu'il n'est pas un outil de trading directionnel, mais un excellent filtre d'incertitude. Pour une stratégie de DCA conditionnel, il n'est pas utile de prédire les hausses parfaites ; il suffit d'avoir un algorithme capable d'affirmer avec une haute précision quand il ne faut *rien* faire.

7.5. Généralisation du modèle

Malgré les résultats peu convaincants de notre modèle gagnant, le backtest sur l'indice d'entraînement paraît encourageant.

Le backtest compare, sur les 269 jours de test, un DCA naïf à 100 €/période avec un DCA conditionnel appliqué uniquement au modèle ATG : 200 € si BUY avec confiance > 55 %, 0 € si SELL avec confiance > 55 %, sinon 100 €. L'alpha est défini comme :

$$\alpha = (\text{valeur finale} + \text{cash non investi}) - \text{valeur DCA naïf} \quad (19)$$

	Cash investi	Valeur équiv.	Alpha	Skip / Double
DCA Naïf (référence)	26 900 €	31 053 €	—	—
ATG conditionnel	26 200 €	31 112 €	+58 €	13 / 6

TABLE 6 – Résultats du backtest DCA conditionnel ATG (seuil 55 %, 100 €/période, 269 jours)

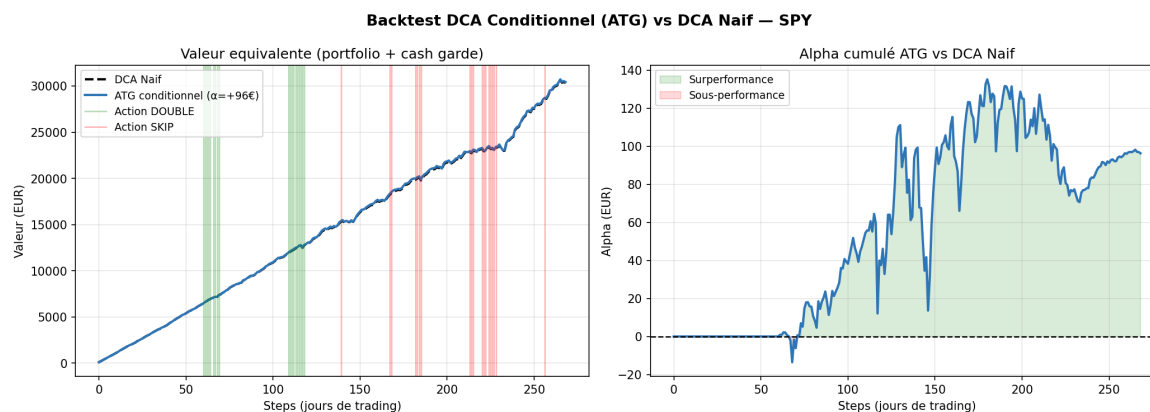


FIGURE 29 – Évolution de la valeur équivalente ATG vs DCA naïf (gauche) et alpha cumulé (droite). Les lignes vertes marquent les signaux DOUBLE, les rouges les signaux SKIP.

L'ATG génère un alpha de +58 € sur 26 200 € investis (+0,22 %) en produisant 6 signaux DOUBLE et 13 SKIP sur la période. Ce résultat doit être lu avec nuance : la progression de l'accuracy avec le seuil de confiance (61 % à > 55 %, 79 % à > 65 %) confirme que le modèle est bien calibré - il sait quand il est fiable. La limite principale reste la **rareté des signaux à haute confiance**, propriété structurelle d'un actif aussi efficient que le SPY.

Je me suis donc lancé dans un test de généralisation du modèle sur d'autres indices.

Tests de généralisation

Le code reprend le modèle déjà entraîné et applique la méthode de backtest précédente à d'autres indices.

Voici les résultats :

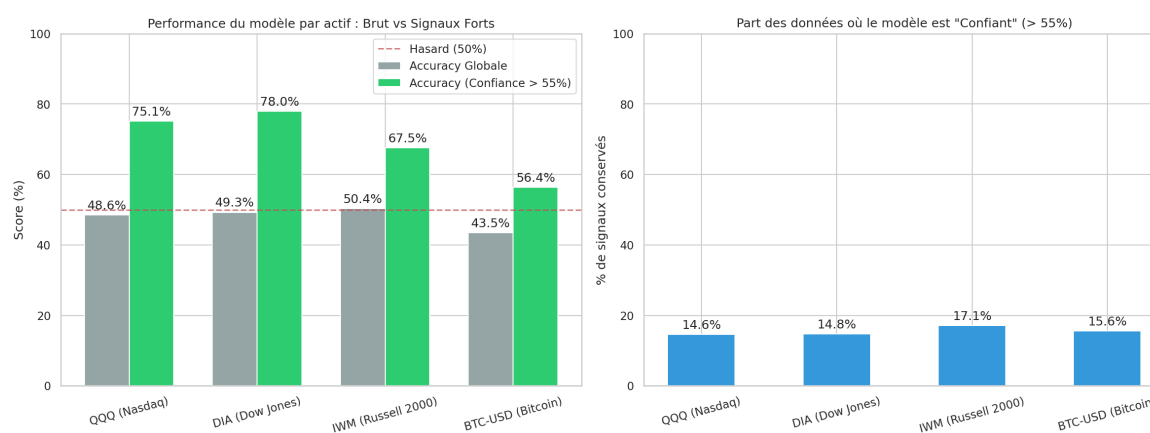


FIGURE 30 – Performance par actif (gauche) et répartition des signaux forts (droite)

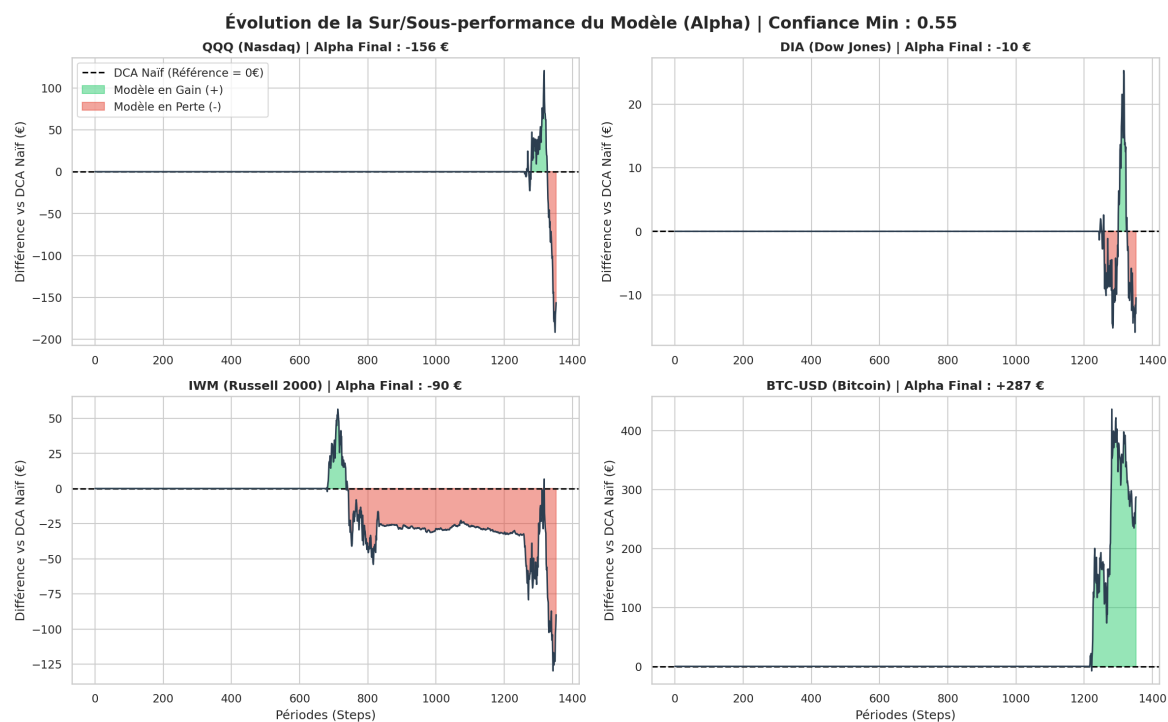


FIGURE 31 – Évolution des profits effectués grâce au modèle ATG entraîné sur le SPY appliqué à 4 indices

Résultats

Bien que le Bitcoin affiche un profit, il est flagrant que le modèle généralise mal. Nous pouvons nous en douter vu les failles structurelles identifiées dans les parties précédentes.

8. Enseignements et pistes d'amélioration

8.1. La limite structurelle des données passées

Les résultats précédents prouvent que le modèle n'a pas toutes les cartes en main pour prédire les cours boursiers futurs. Il semble que les données mathématiques brutes fournies en entrée ne suffisent pas pour que le modèle apprenne vraiment à réfléchir.

Constat fondamental :

Tous les indicateurs techniques classiques (RSI, MACD, moyennes mobiles, volume, etc.) sont des **dérivés du passé**. Par construction, ils ne contiennent aucune information causale sur le futur. Un modèle entraîné uniquement sur ces données peut détecter des corrélations historiques, mais il ne dispose d'aucun signal *prédictif* au sens strict du terme.

Pour qu'un modèle ait une réelle capacité prédictive, il faudrait l'alimenter avec des **données en temps réel** — flux d'actualités, sentiment de marché et données macroéconomiques en direct, ordres d'achat/vente. Cette approche aurait théoriquement plus de chances de fonctionner.

Voici une liste des points que je retiendrai particulièrement de ce défi :

1. **L'importance de valider les hypothèses de base avant de construire.** J'ai passé beaucoup de temps à optimiser un modèle de régression dont le paradigme était fondamentalement limité. Le pivot vers la classification aurait pu intervenir plus tôt ;
2. **Faire attention au data leakage ;**
3. **La puissance du DCA traditionnel.** Comme je l'ai montré dans la partie DCA augmenté, avoir la pleine connaissance de l'évolution future des prix n'augmente pas significativement la performance dans un marché globalement haussier. Ce résultat m'a particulièrement surpris et rappelle que, quoi qu'il arrive, investir régulièrement sans se laisser piéger par le bruit du marché est une stratégie très robuste.
4. **Impact de la taille de la fenêtre.** Ce n'est pas parce que l'on utilise des fenêtres plus grandes donc avec plus d'informations que le résultat sera plus riche ;
5. **Étudier attentivement les métriques à utiliser est essentiel** De la même manière que précédemment, ce n'est pas en ajoutant 20 métriques "économiques" que le modèle devient meilleur. Il faut étudier chacune des métriques et sélectionner celles qui correspondent le plus à l'objectif recherché ;

8.2. Ce que cela implique pour BullEye AI

Cette conclusion ne remet pas en cause le travail réalisé, mais elle en délimite rigoureusement les frontières. Un modèle entraîné exclusivement sur les caractéristiques endogènes et macroéconomiques du SPY conserve une utilité précise :

- Il parvient à modéliser des corrélations statistiques locales entre indicateurs techniques (RSI, MACD, volatilité) et mouvements de prix ;

- Il génère des signaux probabilistes adaptés au comportement historique spécifique de l'indice d'entraînement ;
- Il permet de filtrer efficacement le bruit de marché lorsqu'on lui impose un seuil de confiance strict (DCA conditionnel).

Cependant, son incapacité à s'adapter de manière stable à d'autres indices souligne la spécificité des dynamiques propres à chaque marché. De plus, par construction, un modèle basé sur des séries temporelles passées reste structurellement aveugle aux ruptures de tendance majeures causées par des événements exogènes imprévisibles (chocs géopolitiques, crises de liquidité ou annonces macroéconomiques surprises).

8.3. Pistes d'amélioration identifiées

- Intégrer des **données alternatives** : flux de news financières (NLP sur titres), sentiment Twitter/Reddit, données macroéconomiques en temps réel. Ce point pourrait être effectué à l'aide d'agents, chacun spécialisés dans une thématique et capables de récupérer des données en direct, par exemple à l'aide de Make ;
- Explorer des architectures **Transformer** (ex : Temporal Fusion Transformer [12]) spécialisées pour les séries temporelles. Contrairement à l'ATG, le TFT traite explicitement les features hétérogènes selon leur nature (endogènes, exogènes, statiques), ce qui est directement adapté à notre pipeline mixant indicateurs techniques et données macro. Son mécanisme de *Variable Selection Network* apporterait de surcroît une interprétabilité absente de notre modèle actuel : il serait possible d'identifier quelles variables (VIX, TNX, Fear & Greed) le réseau juge pertinentes à chaque régime de marché.
- Enrichir le backtest avec des métriques financières plus robustes : Sharpe Ratio, Maximum Drawdown, etc ;
- Déployer une interface pour rendre le projet accessible.

9. Conclusion

En plus de 12 mois de travail, j'ai parcouru un cycle complet de recherche appliquée : de la bibliographie à l'implémentation, de l'optimisme des premiers résultats à la découverte des biais structurels du modèle, du pivot stratégique à la remise en question fondamentale.

Je suis très content de voir à quel point j'ai pu dépasser mes premières attentes (ie développer un modèle de deep learning) pour analyser les forces et faiblesses de données d'entrée sur la performance finale d'un modèle.

En conclusion, je dirais que malgré les résultats quelque peu décevants dans l'ensemble puisqu'aucune stratégie efficace n'a été trouvée, je reste persuadé qu'il existe un moyen de perfectionner un modèle récurrent afin de prédire les cours boursiers².

Document rédigé en mai 2026.

2. et ce d'autant plus que durant mes nombreuses recherches je suis tombé sur ce youtubeur [LosingLoonies](#) qui s'est lancé le même défi et semble commencer à avoir des résultats à l'aide d'un LSTM (mais avec une diversité bien plus grande d'indicateurs).

10. Annexes

10.1. Annexe : pipeline générique LSTM

```

1  import numpy as np
2  from tensorflow.keras.models import Sequential
3  from tensorflow.keras.layers import LSTM, Dense, Dropout
4
5  # Creation des sequences
6  forecast_horizon = 7 # On veut predire le prix dans 7 jours
7  window_size = 20
8
9  def create_windows(df, window_size, forecast_horizon,
10     feature_cols):
11     X, y = [], []
12     for i in range(len(df) - window_size - forecast_horizon +
13         1):
14         window = df.iloc[i:i+window_size][feature_cols].
15             values
16         target = df.iloc[i + window_size + forecast_horizon -
17             1]["Close"]
18         X.append(window)
19         y.append(target)
20     return np.array(X), np.array(y)
21
22 X, y = create_windows(df_scaled, window_size,
23     forecast_horizon)
24
25 # Split Temporel Strict
26 split = int(0.95 * len(X))
27 X_train, X_test = X[:split], X[split:]
28 y_train, y_test = y[:split], y[split:]
29
30 # Modele LSTM
31 model = Sequential([
32     LSTM(64, return_sequences=True, input_shape=(window_size,
33         5)),
34     Dropout(0.2),
35     LSTM(32),
36     Dense(1) # Couche de sortie pour la regression du prix
37 ])
38
39 model.compile(optimizer='adam', loss='mse')
40
41 # Entra nement
42 model.fit(X_train, y_train, epochs=20, batch_size=32, verbose
43     =1)
44
45 # Pr diction sur l'ensemble de Test
46 pred_scaled = model.predict(X_test)
47

```

```
41 # Inversion de la normalisation pour retrouver le prix réel
42 y_pred = scaler_y.inverse_transform(pred_scaled)
43 y_test_real = scaler_y.inverse_transform(y_test.reshape(-1,
44     1))
45 # ALIGNEMENT TEMPOREL POUR L’AFFICHAGE (H = 7)
46 # Objectif : faire correspondre visuellement la prédiction
47     faite au jour T avec la réalité du jour T+7
48
49 y_pred_aligned = np.full_like(y_test_real, np.nan)
50 if forecast_horizon <= len(y_pred):
51     y_pred_aligned[forecast_horizon-1:] = y_pred[:len(y_pred)
52         - forecast_horizon + 1]
53
54 # Note : Si on trace plt.plot(y_test_real) et plt.plot(
55     y_pred_aligned), les deux courbes seront désormais
56     parfaitement synchronisées sur l’axe des abscisses (dates
57     ).
```

Listing 4 – Implémentation LSTM avec prédiction à 7 jours et alignement temporel

10.2. Annexe 2 : Benchmark 5 modèles

```

1  import numpy as np
2  import datetime
3  import yfinance as yf
4  from sklearn.preprocessing import MinMaxScaler
5  from sklearn.metrics import mean_squared_error
6  import matplotlib.pyplot as plt
7  from tensorflow.keras.models import Sequential
8  from tensorflow.keras.layers import Dense, Dropout, Flatten,
   LSTM, GRU, Conv1D, MaxPooling1D, GlobalAveragePooling1D,
   Bidirectional
9  from tensorflow.keras.optimizers import Adam
10 from tensorflow.keras.callbacks import EarlyStopping,
   ReduceLROnPlateau
11
12 # Recuperation du ticker avec yahoofinance
13 ticker = input("Entrez le ticker de l'action ou crypto (ex:
   AAPL, TSLA, BTC-USD): ")
14
15 end_date = datetime.date.today()
16 start_date = end_date - datetime.timedelta(days=10*365)
17
18 data = yf.download(ticker, start=start_date, end=end_date,
   progress=False).dropna()
19 data.sort_index(inplace=True)
20 df = data
21
22 # Creation des sequences (comme pr cedeemment)
23 def create_windows(df, window_size, forecast_horizon):
24     X, y = [], []
25     for i in range(len(df) - window_size - forecast_horizon +
   1):
26         window = df.iloc[i:i+window_size][["Open", "High", "Low",
   "Close", "Volume"]].values
27         target = df.iloc[i + window_size + forecast_horizon -
   1]["Close"]
28         X.append(window)
29         y.append(target)
30     return np.array(X), np.array(y)
31
32 window_size = 20
33 forecast_horizon = 1
34 num_features = 5
35
36 X, y = create_windows(df, window_size, forecast_horizon)
37
38 # Normalisation
39 X_scaled = np.zeros_like(X)
40 scalers_X = {}
41 for i in range(num_features):
42     scalers_X[i] = MinMaxScaler()

```

```

43     X_scaled[:, :, i] = scalers_X[i].fit_transform(X[:, :, i])
44
45     scaler_y = MinMaxScaler()
46     y_scaled = scaler_y.fit_transform(y.reshape(-1, 1))
47
48     # Split train/test
49     split = int(0.95 * len(X_scaled))
50     X_train, X_test = X_scaled[:split], X_scaled[split:]
51     y_train, y_test = y_scaled[:split], y_scaled[split:]
52
53     # Dictionnaire des Modeles (5 Modeles ici)
54     models = {}
55
56     # R seau Dense
57     models['Dense-1'] = Sequential([Flatten(input_shape=(
58         window_size, num_features)), Dense(128, activation='relu'),
59         Dropout(0.2), Dense(1)])
60
61     # R seau Convolutif (CNN)
62     models['CNN-1'] = Sequential([Conv1D(32, 3, padding='same',
63         activation='relu', input_shape=(window_size, num_features)),
64         GlobalAveragePooling1D(), Dense(32, activation='relu'),
65         Dense(1)])
66
67     # R seaux R currents Simples et Profonds
68     models['LSTM-1'] = Sequential([LSTM(64, input_shape=(
69         window_size, num_features)), Dense(32, activation='relu'),
70         Dense(1)])
71
72     models['GRU-1'] = Sequential([GRU(64, input_shape=(
73         window_size, num_features)), Dense(32, activation='relu'),
74         Dense(1)])
75
76     # Architectures Hybrides (CNN + RNN)
77     models['CNN-LSTM'] = Sequential([Conv1D(32, 3, padding='same',
78         activation='relu', input_shape=(window_size, num_features)),
79         MaxPooling1D(2), LSTM(64), Dense(32, activation='relu'),
80         Dense(1)])
81
82     # Entra nement et benchmark + RMSE
83     results = {}
84
85     for name, model in models.items():
86         print(f"== Entra nement du mod le {name} ==")
87
88         model.compile(optimizer=Adam(0.001), loss='mse', metrics
89             =['mae'])
90         early_stop = EarlyStopping(monitor='val_loss', patience
91             =10, restore_best_weights=True)
92         reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor
93             =0.5, patience=5, min_lr=1e-5)

```

```

79
80     history = model.fit(
81         X_train, y_train,
82         epochs=100,
83         batch_size=32,
84         validation_data=(X_test, y_test),
85         callbacks=[early_stop, reduce_lr],
86         verbose=0
87     )
88
89     y_pred_scaled = model.predict(X_test)
90     y_pred = scaler_y.inverse_transform(y_pred_scaled)
91     y_test_real = scaler_y.inverse_transform(y_test)
92
93     y_pred_aligned = np.full_like(y_test_real, np.nan)
94     if forecast_horizon <= len(y_pred):
95         y_pred_aligned[forecast_horizon-1:] = y_pred[:len(
96             y_pred)-forecast_horizon+1]
97
98     # Calcul du Root Mean Squared Error
99     rmse = np.sqrt(mean_squared_error(y_test_real[
100         forecast_horizon-1:], y_pred_aligned[forecast_horizon
101         -1:]))
102     results[name] = rmse
103
104     # Graphe individuel optionnel
105     # plt.figure(figsize=(12,4))
106     # plt.plot(y_test_real, label='Prix r el')
107     # plt.plot(y_pred_aligned, label=f'Prix pr dit ({name})
108     #         ')
109     # plt.title(f'{ticker} - {name} - RMSE={rmse:.2f}')
110     # plt.legend()
111     # plt.show()
112
113     # Graphique comparatif RMSE
114     plt.figure(figsize=(12,5))
115     names = list(results.keys())
116     values = [results[n] for n in names]
117     plt.barh(names, values, color='skyblue')
118     plt.xlabel("RMSE")
119     plt.title(f"Comparaison des RMSE pour {ticker}")
120     plt.gca().invert_yaxis()
121     plt.grid(axis='x')
122     plt.show()
123
124     # lection du meilleur mod le
125     best_model_name = min(results, key=results.get)
126     print(f"Meilleur mod le : {best_model_name} avec RMSE = {
127         results[best_model_name]:.4f}")
128
129     best_model = models[best_model_name]

```

```
125 last_window = X_scaled[-1].reshape(1, window_size,
    num_features)
126 pred_scaled = best_model.predict(last_window)
127 pred_close = scaler_y.inverse_transform(pred_scaled.reshape
    (-1,1))[0][0]
128
129 print(f"Prix pr dit pour {ticker} dans {forecast_horizon}
    jours : {pred_close:.2f}")
```

Listing 5 – Script de Benchmark de 5 modèles de Deep Learning pour la prédiction boursière

10.3. Annexe 3 : dictionnaire détaillé des 17 features du pipeline final

Afin d'alimenter les modèles de Machine Learning (notamment l'architecture ATG) avec des données pertinentes et stationnaires, l'historique brut des actifs (Open, High, Low, Close, Volume) a été transformé en 17 indicateurs spécifiques. Ces *features* ont été normalisées de manière intrinsèque avant même le passage dans le `RobustScaler`, afin d'assurer leur invariance par rapport au prix absolu de l'actif traité.

Elles se divisent en 5 catégories fondamentales :

1. Dynamique des rendements et Momentum (4 features)

Cette catégorie capte la direction et l'accélération de la tendance sur différentes échelles de temps.

- **returns** : Rendement journalier classique (variation en pourcentage du prix de clôture par rapport à la veille).
- **momentum_3, momentum_7, momentum_14** : Rendements cumulés sur des fenêtres glissantes de 3, 7 et 14 jours. Ils informent le modèle sur la tendance à très court, court et moyen terme, permettant de détecter les phases d'essoufflement ou d'accélération d'un mouvement.

2. Oscillateurs et Indicateurs de Tendance (4 features)

Ces indicateurs classiques de l'analyse technique ont été mis à l'échelle pour être bornés ou ramenés à des pourcentages.

- **RSI** (Relative Strength Index) : Divisé par 100 pour être borné entre 0 et 1. Mesure la magnitude des variations de prix récentes pour évaluer les conditions de surachat ou de survente.
- **MACD** (Moving Average Convergence Divergence) : Divisé par le prix de clôture (**Close**). Le MACD brut étant exprimé dans la devise de l'actif, cette division le convertit en pourcentage, le rendant comparable entre un indice valant 500\$ et un actif valant 60 000\$.
- **STOCH** (Oscillateur Stochastique) : Divisé par 100. Indique la position du prix de clôture actuel par rapport à son range (plus haut / plus bas) sur les 14 derniers jours.
- **ADX** (Average Directional Index) : Divisé par 100. Il ne mesure pas la direction, mais la *force* absolue de la tendance en cours. Un ADX élevé indique une tendance forte (haussière ou baissière), un ADX faible indique un marché en *range*.

3. Volatilité et Structure spatiale du prix (4 features)

Ces variables aident le réseau à comprendre la nervosité du marché et la morphologie intrajournalière des bougies (chandeliers japonais).

- **Volatility** : Écart-type glissant sur 10 jours de la feature **returns**. C'est la mesure directe du risque et de la nervosité récente du marché.
- **BB_width** (Largeur des Bandes de Bollinger) : Différence entre la bande supérieure et inférieure, divisée par le prix de clôture. Identifie les phases de compression (souvent suivies de cassures violentes) et les phases d'expansion de la volatilité.

- **high_low_range** : Amplitude de la séance (*High* – *Low*) normalisée par le prix de clôture. Mesure l'amplitude des combats intrajournaliers entre acheteurs et vendeurs.
- **close_position** : Position relative de la clôture au sein du range journalier (valeur entre 0 et 1). Une valeur de 1 signifie que l'actif a clôturé à son plus haut (pression acheteuse maximale), tandis que 0 indique une clôture au plus bas. L'ajout de 10^{-9} au dénominateur empêche les erreurs de division par zéro lors des jours de cotation "plats".

4. Dynamique des Volumes (2 features)

Le volume est le carburant des mouvements boursiers. Un mouvement de prix sans volume est souvent un faux signal.

- **volume_change** : Variation en pourcentage du volume échangé par rapport à la veille.
- **volume_ma_ratio** : Ratio entre le volume du jour et sa moyenne glissante sur 20 jours. Une valeur de 2 ou 3 indique une anomalie d'activité majeure, souvent associée à des annonces institutionnelles ou des paniques.

5. Données Macroéconomiques et Sentiment (3 features exogènes)

Il s'agit de la véritable valeur ajoutée du modèle final : l'intégration de données qui ne dérivent pas directement du prix de l'actif, apportant un contexte global (et partiellement causal).

- **vix** (CBOE Volatility Index) : Surnommé l'"indice de la peur". Il mesure la volatilité implicite attendue par le marché des options sur le S&P 500 à 30 jours.
- **tnx** (Taux obligataire US à 10 ans) : Le taux sans risque de référence. Une hausse brutale du TNX provoque généralement une rotation sectorielle au détriment des actifs risqués (actions technologiques, cryptomonnaies).
- **fear_greed** : Indice composite du sentiment de marché (CNN) mesurant si les investisseurs sont dominés par la peur extrême (sous-évaluation potentielle) ou l'avidité (risque de bulle/correction).

Références

- [1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available : <https://deeplearning.cs.cmu.edu/S23/document/readings/LSTM.pdf>
- [2] S. Yang, X. Yu, and Y. Zhou, “LSTM and GRU neural network performance comparison study : Taking yelp review dataset as an example,” in *Proceedings of the 2020 International Workshop on Electronic Communication and Artificial Intelligence (IWECAI)*. IEEE, 2020, pp. 98–101. [Online]. Available : https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9221727&casa_token=DUi6UpaaKNsAAAAA:FtasLY40TYQLKj1n000qiUMzSfly6HnpB_KdEpD4DOQQpWf2d26avt_Uhfl_9o6W0HKL9BZrw8_6&tag=1
- [3] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv :1412.3555*, 2014. [Online]. Available : <https://arxiv.org/pdf/1412.3555.pdf>
- [4] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv :1803.01271*, 2018. [Online]. Available : <https://arxiv.org/abs/1803.01271>
- [5] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “Statistical and machine learning forecasting methods : Concerns and ways forward,” *PLOS ONE*, vol. 13, no. 3, p. e0194889, 2018. [Online]. Available : <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0194889>
- [6] S. Ben Taieb, G. Bontempi, A. F. Atiya, and A. Sorjamaa, “A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition,” *Expert Systems with Applications*, vol. 39, no. 8, pp. 7067–7083, 2011, revue et comparaison empirique sur 111 séries (NN5) : stratégies Recursive, Direct, MIMO, DirRec, DIRMOMO. Les résultats sont nuancés selon les datasets - la supériorité de Direct n'est pas universelle. [Online]. Available : https://www.sciencedirect.com/science/article/pii/S0957417412000528?casa_token=2iO0B24Z9qIAAAAA:FzNrU-yLahbz9tNar4w5fEtiAjOJeMqkAm_sm_78NP9grHAE_CVYPOt0FaeLctG49GAK4WR0Z7d_
- [7] S. B. Taieb, A. Sorjamaa, and G. Bontempi, “Long-term prediction of time series by combining direct and MIMO strategies,” in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2010.
- [8] M. M. Kumbure, C. Lohrmann, P. Luukka, and J. Porras, “Machine learning techniques and data for stock market forecasting : A literature review,” *Expert Systems with Applications*, vol. 197, p. 116659, 2022, survey sur 150+ études : les RNN surpassent en moyenne les feedforward NN sur séries financières, confirmant l'existence de dépendances temporelles non capturées par MLP. [Online]. Available : <https://arxiv.org/pdf/1906.07786>
- [9] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning internal representations by error propagation,” California Univ San Diego La Jolla Inst for Cognitive Science, Tech. Rep. ICS-8506, 1985. [Online]. Available : <https://apps.dtic.mil/sti/tr/pdf/ADA164453.pdf>
- [10] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.

- [11] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE transactions on neural networks*, vol. 5, no. 2, pp. 157–166, 1994. [Online]. Available : https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=279181&casa_token=3n4pqw1sJS0AAAAA:EDr2pS7KUFE7g9JSzn5R9h__oyu0CIq43BZxghvAVBE0tp6v3NwwGTZbvIwjT0Q64XZn7VMxwRiaG&tag=1
- [12] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.